CURRICULUM HANDBOOK & TEACHING GUIDE

Digital Literacy & Computer Education Programme

A Complete Multi-Phase Curriculum for Secondary School Students (Ages 12-18)
Assuming Absolutely Zero Prior Computer Experience

Developed by

Dzuels Foundation

Version 1.0 | Academic Year 2026-2027

Table of Contents

1. Executive Summary & Curriculum Philosophy

2. Teaching Guide & Classroom Management

3. Assessment Framework & Evaluation Systems

4. Phase 1: BASIC (Foundational Literacy)

- Module 1: Introduction to Computers
- Module 2: Parts of a Computer
- Module 3: Computer Usage & Core Terminology
- Module 4: Operating Systems & Basic Configurations
- Module 5: File Management & Organization
- Module 6: Internet Basics & Digital Citizenship
- Module 7: Microsoft Office Productivity Suite

5. Phase 2: INTERMEDIATE (Creative Logic & App Development)

- Module 8: Computational Thinking & Algorithmic Logic
- Module 9: Scratch Visual Block-based Programming
- Module 10: Python Scripting Language Basics
- Module 11: JavaScript Concepts & Web Logic
- Module 12: Mobile App Development (React Native Concepts)

6. Phase 3: ADVANCED (Specialized Technologies & Futures)

- Module 13: HTML & CSS Web Design & Responsive Layouts
- Module 14: Version Control (Git & GitHub Collaboration)
- Module 15: Artificial Intelligence & Responsible Prompting
- Module 16: Robotics & Physical Computing (Arduino Framework)
- Module 17: Digital Creativity (Graphic Design & Basic Media Editing)
- Module 18: Cybersecurity & Digital Hygiene Basics
- Module 19: Career Pathways & Next Steps

7. Sample Lesson Plans

8. Recommended Resources & Free Software

9. Appendices & Classroom Templates

- Appendix A: Glossary of Core Computer Science Terms

- Appendix B: Essential Keyboard Shortcuts Cheatsheet
- Appendix C: Computer Laboratory Rules Poster
- Appendix D: Internet Safety & Cybersecurity Rules
- Appendix E: Digital Citizenship Pledge
- Appendix F: Student Progress Tracking Template
- Appendix G: Laboratory Attendance Log Template
- Appendix H: Rubric Grading Template
- Appendix I: Project Evaluation Template
- Appendix J: Sample Certificate of Completion Structure
- Appendix K: Dzuels Foundation Contact Information

1. Executive Summary & Curriculum Philosophy

Program Mission & Vision

Vision: To alleviate poverty through quality education.

Mission: To make quality education accessible to every child within reach.

The Dzuels Foundation Digital Literacy & Computer Education Programme is built directly on these founding principles. By providing secondary school students (ages 12–18) in Ekiti State and beyond with a structured, international-standard education in computer science, we aim to bridge the digital divide. Assuming absolutely no prior technology experience, this multi-phase curriculum provides a clear path to technological self-sufficiency, logical problem-solving, and future-ready employment.

Core Foundation Values (D-R-N-I-C-E-R)

The curriculum is designed to not only teach technological skills but also to cultivate character and leadership. The program instills the Dzuels Foundation core values of Discipline, Responsibility, No-Excuse, Integrity, Compassion, Excellence, and Respect in every classroom activity, laboratory session, and project delivery.

- Discipline (D): Dedication to structured study and consistent laboratory practice.
- Responsibility (R): Taking ownership of one's learning and caring for shared laboratory hardware.
- No-Excuse (N): Overcoming infrastructure challenges through persistence, problem-solving, and simulation-based tools.
- Integrity (I): Upholding academic honesty, proper citation of research and AI tools, and collaboration ethics.
- Compassion (C): Supporting classmates through peer mentoring and cooperative team learning.
- Excellence (E): Striving for the highest quality in code structures, slide design, and project execution.
- Respect (R): Demonstrating positive digital citizenship, online safety habits, and respectful communication.

Curriculum Philosophy & Pedagogical Framework

Our educational philosophy centers on active, constructive learning. We believe that technology literacy is not acquired by watching, but by doing. Thus, this curriculum is heavily project-based, dedicating more than 60% of lesson time to hands-on keyboard, mouse, or code manipulation.

We balance three primary dimensions of learning:
1. Conceptual Theory: Building computational thinking, vocabulary, and structural understanding.
2. Practical Application: Mastering physical coordination (posture, speed) and software tooling.
3. Creative Synthesis: Encouraging design, logical problem-solving, and original code through individual and group projects.

By grounding students first in absolute basics (Phase 1), moving to programming logic (Phase 2), and finalizing with specialized modern technologies like Web Design, Git, and AI prompting (Phase 3), we ensure a gradual, non-intimidating transition into digital mastery.

2. Teaching Guide & Classroom Management

This teaching guide outlines classroom instructions, laboratory configurations, and pacing adjustments necessary to implement this curriculum effectively across diverse school structures.

Teacher Preparation Protocol

Effective computer science education begins before students enter the lab. Teachers must perform the following preparation routines before every lesson:
1. Hardware Checks: Ensure all computer workstations are powered, mouse devices are connected, and keyboards are clean.
2. Software Configuration: Confirm that necessary software (Thonny, Scratch, Browsers, Office) is installed and updated on all student computers.
3. File Distribution: Pre-load any required exercise files or templates on student desktop paths or cloud drives.
4. Teacher Demonstration setup: Ensure the projector or screen display is functional and readable from the back of the room.
5. Backup Plans: Have offline worksheets and physical activities ready in case of power outages or internet failures.

Classroom and Computer Lab Management

Classroom management in a computer lab requires unique strategies due to the physical distractions of screens:
1. The 'Screens Down' Rule: Establish a clear signal (e.g. ringing a bell or saying 'Screens Down') where students must close laptop lids or look away from monitors to focus on teacher instructions.
2. Monitored Workspaces: Walk around the room constantly. Do not remain at the teacher's desk. Position screens so they are visible from a central walking path if possible.
3. Collaborative Pairing: If computers are limited, pair students as 'Driver' (typing) and 'Navigator' (reviewing logic). Have them swap roles every 15 minutes.
4. Safe Environment: Post rules clearly regarding food, drinks, and cable handling. Ensure students pack cables neatly after class.

Lesson Pacing and Duration

Recommended Lesson Duration:
- Total Time: 45 Minutes per lesson.
- Introduction & Concept explanation: 10 Minutes.
- Teacher Demonstration: 5 Minutes.
- Student Hands-on Lab Activity: 25 Minutes.
- Clean-up & Wrap-up: 5 Minutes.

Double periods (90 Minutes) are recommended for complex coding tasks in Python, JavaScript, and Web Design to allow deeper focus.

Differentiated Instruction for Varying Speeds

Computer science classes often have a wide range of learning speeds. Manage this through differentiated instruction:

1. Extension Tasks: For fast learners, provide 'Stretch Goals' (e.g. add a high score screen in Scratch, add styling rules in CSS, write nested loops in Python).
2. Peer Mentoring: Assign advanced students who complete tasks early as 'Student Helpers' or peer tutors to support classmates. This solidifies their own understanding.
3. Simplified Tasks: For struggling learners, focus on core objectives only (e.g. modify existing code variables rather than writing a program from scratch).
4. Visual & Audio Aids: Provide printed, step-by-step screenshot sheets for students who struggle with verbal instructions.

Inclusive & Accessible Teaching Strategies

Ensure the computer lab is inclusive to all backgrounds and learning profiles:

1. Language Adaptation: Use simple, clear English and write key terminology on the board. Avoid overly technical jargon when starting.
2. Gender Balance: Actively encourage female students to lead programming and engineering tasks. Set balanced group structures.
3. Ergonomic Support: Provide mouse devices suitable for left-handed students. Adjust seat heights and monitor brightness to prevent strain.
4. Neurodiversity: Allow students who get overstimulated by lab noises (keyboard tapping, fans) to wear headphones during silent working blocks.

3. Assessment Framework & Evaluation Systems

To measure skills growth accurately, the program utilizes a continuous, multi-dimensional assessment system. This balances baseline checks, over-the-shoulder feedback, structured tests, and portfolio milestones.

Diagnostic Assessment

Diagnostic Assessment (Baseline):
Conducted before starting a phase or module. This consists of simple surveys, interactive discussions, or high-level questionnaires to check baseline technology exposure. For complete beginners, it checks if they know how to identify a monitor, mouse, or if they have used mobile devices before.

Formative Assessment

Formative Assessment (Ongoing feedback):
Incorporated throughout every lesson. Teachers check progress through:
- 'Exit Tickets': Short, 2-question slips turned in at the end of class checking core concepts.
- Over-the-shoulder checks: Reviewing code structures and typing styles during practical exercises.
- Think-Pair-Share: Asking students to discuss a programming problem with their neighbor and report back.

Summative Assessment

Summative Assessment (Final grading):
Conducted at the end of each module and phase. This includes:
- Conceptual Written Exams: Checking vocabulary, short answer logic, and syntax analysis.
- Practical Performance Exams: Timed laboratory tasks where students build files, folders, or scripts under observation.

Project-Based Evaluation

Project-Based Assessment:
Evaluates the synthesis of multiple skills. Individual and Group projects require students to apply their cumulative knowledge to solve real-world problems. Grading is based on a standard rubric assessing functionality, structure, aesthetics, and code logic.

Peer & Self Evaluation

Peer & Self Assessment:
Encourages metacognition and collaboration. Students review each other's code structures during peer-review blocks, checking readability and commenting. They also complete self-evaluation forms to reflect on their learning progress and identify personal gaps.

Cumulative Portfolio Assessment

Portfolio Assessment:
The ultimate evaluation tool. Students assemble all their completed works (Word files, Excel spreadsheets, Scratch links, Python scripts, and responsive website code) into a single folder or a remote Git repository. The portfolio showcases their growth from absolute beginners to functional software developers.

4. Phase 1: BASIC (Foundational Literacy)

Introduction & Context

Welcome to Phase 1 (BASIC) of the Digital Literacy & Computer Education Programme. This phase is designed specifically for students aged 12 to 18 who have absolutely no prior experience with technology. The curriculum assumes that students have never touched a computer, seen a desktop interface, or held a mouse. The pedagogical focus of this phase is to demystify the computer, build confidence, develop physical coordination (such as typing posture and mouse grip), and introduce fundamental digital tools that are essential for modern education and future careers.

By the end of this phase, students will transition from absolute beginners to confident users of basic computer systems, operating systems, file management concepts, internet safety, and Microsoft Office tools.

Phase Description

Phase 1 serves as the critical foundation of the entire curriculum. It spans 16 weeks, comprising 32 lessons of 45 minutes each. The content begins with conceptual understanding (What is a computer? Where is it used?) and gradually moves to physical interactions (parts of a computer, keyboard layouts, mouse movements). As students gain confidence, they are introduced to software applications, starting with Operating Systems (Windows/Linux/macOS), file organization, safe web browsing, and introductory document creation and data processing using Microsoft Office. Every lesson balances simple explanations with hands-on practical exercises to reinforce learning.

Learning Objectives & Core Outcomes

Learning Objectives:

- Identify and describe the main hardware components of desktop and laptop computers.
- Perform essential physical operations, including correct typing posture, keyboard typing, and mouse actions (clicking, dragging, scrolling).
- Navigate operating system environments, locate applications, configure settings, and handle basic system maintenance.

- Establish effective file management structures, including creating, organizing, copying, moving, and securing files and folders.
- Understand the basic principles of the internet, perform safe search queries, and practice digital citizenship.
- Create, edit, format, and save documents in Microsoft Word, spreadsheets in Excel, and presentation slides in PowerPoint.

Expected Learning Outcomes:

- Students can explain the function of the CPU, RAM, storage devices, and peripherals in simple terms.
- Students demonstrate smooth physical control over the mouse and keyboard using correct posture.
- Students can boot, log in, navigate the desktop, launch applications, and shut down a computer safely.
- Students demonstrate the ability to organize digital folders, manage file hierarchies, and use storage drives.
- Students identify cybersecurity risks (such as insecure sites) and demonstrate safe online communication habits.
- Students create an academic report in Word, a basic monthly budget spreadsheet in Excel, and an educational presentation in PowerPoint.

Core Skills Students Will Gain

- Hardware identification and setup
- Fine motor skills for mouse control
- Home-row typing and touch-typing basics
- Operating system navigation and customization
- Logical file structures and directory management
- Safe web browsing and digital research methods
- Document formatting and styling
- Basic arithmetic formulas and chart generation
- Effective visual slide design and delivery

Weekly Schedule

Weeks	Module Title	Topics Overview
1-2	Module 1: Introduction to Computers	Definition, history, types, impact on society, benefits of digital literacy.
3-4	Module 2: Parts of a Computer	Physical identification, peripherals (monitor, keyboard, mouse, CPU, printers, storage).
5-6	Module 3: Computer Usage	Mouse actions, keyboard layouts, typing posture, turning computer on/off, desktop terms.
7-8	Module 4: Operating Systems	Windows, Linux, macOS comparison, File Explorer, settings, installing apps, basic troubleshooting.
9-10	Module 5: File Management	Creating/renaming directories, copying/moving files, file extensions, USBs, and cloud storage basics.
11-12	Module 6: Internet Basics	Browsers, search engines, safe search, cyber hygiene, strong passwords, email etiquette.
13-16	Module 7: Microsoft Office Suite	Intro to Word (document styling), Excel (cells, formulas, charts), PowerPoint (slides, transitions), Drive/PDF.

Recommended Teaching Methods & Classroom Activities

Teaching Methods:

- Direct Instruction with Physical Demonstration: Teachers show physical hardware before students interact.
- Guided Practice: Teachers walk students through computer operations step-by-step using a projector.
- Kinesthetic Learning: Direct hands-on laboratory work where students touch, type, and click.
- Peer-Assisted Learning: Pairing students of differing progression speeds to support collaboration.
- Gamified Learning: Using interactive typing games (e.g., Tux Typing) to make practice engaging.

Classroom Activities:

- Hardware Scavenger Hunt: Students find and label physical computer components in the lab.
- Typing Posture Audits: Students work in pairs to check and correct each other's typing posture.
- Desktop Customization: Personalizing wallpaper, window colors, and organizing desktop icons.
- File Sorting Race: Organizing a disorganized folder of mixed files into designated, structured sub-folders.
- Safe Browsing Mock Scenarios: Interactive discussions identifying phishing emails or safe vs. unsafe websites.

Hands-on Practical Activities

- System Boot & Shutdown Sequence: Booting, logging in, navigating the desktop, and performing a safe shutdown.
- Text Document Creation: Writing a 1-page biography using proper margins, fonts, and headers in Word.

- Personal Expense Log: Designing a personal budget table with automatic SUM and AVERAGE formulas in Excel.
- My Hobby Presentation: Developing a 5-slide presentation with transitions and images in PowerPoint.

Projects Framework

Individual Project:

My Digital Profile: Students create a personal folder on the computer. Inside this folder, they write a Word document describing their background, insert an Excel spreadsheet representing their weekly school schedule, and save a PowerPoint presentation introducing their family. The entire folder is then copied to a USB drive or uploaded to Google Drive to demonstrate complete digital storage mastery.

Group Project:

The Computer Lab Guide: Working in groups of three, students design a physical poster or a digital guide detailing rules for the computer lab, safe internet usage guidelines, and keyboard shortcuts. They write the text together, format it using Word or PowerPoint, print it out, and present it to the class.

Teacher Guidance & Lab Notes

> **Facilitator Pacing Note**
>
> *Teacher Note: Complete beginners may display extreme hesitation or fear of 'breaking' the computer. Establish a welcoming, low-stakes environment. Encourage exploration. Emphasize that clicking the wrong button will not damage the hardware. Pacing is critical; do not rush mouse control and typing posture lessons, as bad habits are difficult to correct later.*

Required Physical & Digital Resources

- Computers (Desktop or Laptop) with operating systems installed
- Projector or Large screen for classroom demonstrations
- Physical disassembled computer parts (old motherboard, RAM sticks, hard drives) for physical inspection

- Tux Typing or Mavis Beacon typing software (free/open-source)
- USB Flash drives for file transfer exercises
- Internet connection (optional but recommended for Modules 6 and 7)

Assessment Methods & Framework

- Practical Laboratory Checkpoints: Direct observation of student capabilities (e.g., opening applications, copying files).
- Short Weekly Quizzes: Conceptual checks on vocabulary (e.g., What is hardware vs. software?).
- Project Rubrics: Detailed scoring for individual and group projects.
- End-of-Phase Written & Practical Exam: 50% theoretical multiple choice, 50% hands-on tasks in the lab.

Grading Rubrics

Component	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Hardware Identification	Correctly identifies all physical parts and explains their core functions.	Identifies most parts correctly; explains basic functions.	Identifies main parts (monitor, mouse) but confuses internal parts.	Cannot identify basic hardware parts.
Computer Operations	Performs boot, login, navigation, and shutdown smoothly and safely.	Performs operations with occasional prompts or minor hesitations.	Requires constant supervision to perform boot and shutdown.	Unable to turn the computer on or off independently.
Typing &	Maintains	Good posture;	Uses index	Incorrect

Posture	excellent posture; types with two hands on home row; high accuracy.	uses multiple fingers; moderate typing speed.	fingers only ('hunt and peck'); poor posture.	posture; unable to type basic sentences.
File Management	Organizes nested directories; copies, moves, and deletes files with ease.	Creates folders; copies and moves files; minor organization errors.	Can create folders but struggles with copying/moving files.	Cannot create folders or organize files.
Microsoft Office	Formats documents professionally; uses formulas in Excel; designs clean slides.	Creates basic Word, Excel, and PowerPoint files with minor design flaws.	Struggles with formatting, formulas, or slide transitions.	Unable to use Microsoft Office applications.

End-of-Phase Examination Framework

Phase 1 Final Examination Structure:
Part A: Theory (30 Marks)
- 20 Multiple Choice Questions (definitions, hardware parts, operating systems, internet safety)
- 5 Matching Questions (keyboard shortcuts to actions)
- 5 Short Answer Questions (e.g., Explain the difference between copying and moving a file)

Part B: Practical (70 Marks)
- Task 1: Boot computer, create a folder named '[Student Name]_Final', and create two subfolders: 'Text' and 'Data'. (15 Marks)
- Task 2: Open Word, type a paragraph of 5 sentences about internet safety, bold the title, format font to Calibri 12pt, insert a 2x3 table listing 3 safety rules, and save as 'rules.docx' in the 'Text' subfolder. (25 Marks)
- Task 3: Open Excel, create a table with columns: 'Item Name', 'Quantity', 'Unit Price', 'Total

Price'. Enter 3 items, calculate Total Price using formulas, calculate Grand Total using SUM, and save as 'expenses.xlsx' in the 'Data' subfolder. (20 Marks)
- Task 4: Safely copy the entire parent folder to a USB drive or upload to a Google Drive path. Close all windows, log out, and safely shut down the computer. (10 Marks)

Certification Requirements

Certification of Basic Digital Literacy:
To be awarded the Certificate of Basic Digital Literacy, a student must meet the following criteria:
1. Minimum attendance of 85% in all Phase 1 classroom and laboratory sessions.
2. Submission of the Individual Project ('My Digital Profile') and the Group Project ('Computer Lab Guide') with a passing grade (50% or above on the rubric).
3. Achieve a minimum score of 50% on the End-of-Phase Examination (Written and Practical combined).

Module 1: Introduction to Computers

Module Overview

This module sets the stage for the entire curriculum by introducing students to the computer. Students will learn what a computer is, trace its history in simple terms, discover the different types of computers around them, and explore how computers impact modern society. The primary goal is to remove any fear of computers and build interest.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Define the term 'computer' in simple, understandable terms.
- Outline a brief history of computers (from abacus to mechanical calculators to microcomputers).
- Identify the different types of computers (desktops, laptops, tablets, smartphones).
- State the benefits of computer literacy in daily life and careers.

Expected Learning Outcomes:

- Students can explain that a computer is a machine that processes input to produce output.
- Students can name at least three historical counting devices.
- Students can point out different types of computers in their environment.
- Students state why computer skills are essential for their future.

Topics Covered

- What is a computer? Input, Processing, Output, Storage (IPOS cycle).
- A short history: Early counting (abacus, Napier's bones, Pascaline, Analytical Engine) to modern microchips.
- Types of computers: Personal Computers (Desktops & Laptops), Mobile Devices (Smartphones, Tablets), Supercomputers.
- Where computers are used: Schools, hospitals, banks, supermarkets, homes.

- Importance of computer literacy: Communication, career advancement, research, problem solving.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Interactive demonstration: Teacher turns on a computer, inputs numbers via keyboard, showing how the computer calculates and displays them.
- Computer Classification: Students look at pictures of devices and classify them (Desktop, Laptop, Smartphone, Tablet).
- Field trip/walk: A walk around the school or local offices to observe where computers are placed and how they are used.

Student Activities:

- Drawing a diagram of the IPOS (Input-Processing-Output-Storage) cycle using paper and crayons.
- Brainstorming in pairs: Write down 5 activities that would be impossible without computers.

Teacher Guide & Notes

Start by showing a physical calculator and a computer. Compare how they process inputs. Use analog stories (like baking a cake: ingredients = input, baking = processing, cake = output) to explain the Input-Processing-Output cycle. Keep historical details high-level: avoid dates, focus on evolution.

Exercises & Review Questions

Exercises:

- Fill in the blanks matching 'Input', 'Processing', 'Output' with real-world scenarios.
- Identify whether a given device (e.g. tablet, banking machine) is a computer.

Review Questions:

- What are the four main steps of the computer data cycle?

- List three types of computers people use every day.
- Why is a smartphone considered a computer?

Projects & Assessments

Mini Project:

Draw Your Dream Computer: Students design and draw their own futuristic computer, labeling where input is entered and where output is shown.

Assessment Method:

10-question multiple-choice quiz covering computer definitions, types, and areas of application.

Home Assignment:

Ask parents or guardians what type of computers they use at work or home, and write a 3-sentence summary of their responses.

Required Materials

Board, markers, printed pictures of historic and modern computers, real smartphone, tablet, and desktop.

Module 2: Parts of a Computer

Module Overview

In this module, students learn to identify and describe the physical parts of a computer system. They will learn to distinguish between input devices (keyboard, mouse, scanner, webcam), output devices (monitor, printer, speakers), processing units (CPU), storage devices, and understand how they work together.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Identify the main physical parts of a computer system.
- Distinguish between input and output devices.
- Explain the role of the CPU (Central Processing Unit) in simple terms.
- Identify standard storage media (hard drives, solid-state drives, USBs).

Expected Learning Outcomes:

- Students physically point out the monitor, mouse, keyboard, CPU box, printer, speakers, and webcam.
- Students categorize list of devices into input, output, or storage.
- Students describe the CPU as the 'brain' of the computer.
- Students explain what a USB flash drive is used for.

Topics Covered

- Basic components of a desktop system: Monitor, Keyboard, Mouse, System Unit (CPU).
- Input peripherals: Keyboard, Mouse, Scanner, Webcam, Microphone.
- Output peripherals: Monitor (Screen), Printer, Speakers, Headphones.
- The System Unit: Motherboard, CPU, RAM (Memory), Storage (HDD/SSD).
- Storage media: Internal hard drives, External hard drives, USB Flash Drives, Memory Cards.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Labeling Lab: Students go around the computer lab placing sticky notes with correct names on hardware components.
- Hardware Inspection: Handling a disassembled motherboard, RAM stick, and old hard drive (safely, disconnected from power) to see internal components.

Student Activities:

- Completing a worksheet where they draw lines connecting the name of a hardware component to its picture.
- Sorting activity: Placing physical cards representing devices (e.g. Printer, Mouse) into 'Input' and 'Output' baskets.

Teacher Guide & Notes

Ensure safety first. Emphasize that internal parts should only be handled when powered down. Use the analogy of the human body: CPU is the brain, monitor is the face/expressions, keyboard/mouse are hands/senses. Teach students to treat hardware gently: no slamming keys or pulling cables.

Exercises & Review Questions

Exercises:

- Multiple choice: Which of these is an input device? (a) Monitor (b) Keyboard (c) Speaker
- True or False: The CPU stores all our files permanently.

Review Questions:

- What is the full name of the CPU, and why is it important?
- What is the difference between an input device and an output device?
- Name two devices used for storing digital files.

Projects & Assessments

Mini Project:

Hardware Scrapbook: Cut out pictures of computer components from old magazines or newspapers, paste them in a notebook, and write a brief description of what each part does.

Assessment Method:

Practical test where the teacher points to a hardware component and the student must state its name and function.

Home Assignment:

Draw a sketch of a complete computer system setup on a piece of paper, including a keyboard, mouse, monitor, CPU, and speakers.

Required Materials

Decommissioned computer system units, old RAM and hard drive parts, sticky notes, markers, worksheets.

Module 3: Computer Usage

Module Overview

This module moves students from passive observation to active computer interaction. It teaches proper typing posture, mouse grips, double-clicking, drag-and-drop, desktop layouts, and the correct processes for powering a computer on, logging in, logging out, and shutting down.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Operate a mouse using correct finger placements for left-click, right-click, double-click, and scrolling.
- Perform basic navigation using a laptop touchpad.
- Demonstrate proper keyboard posture and finger placement (home row).
- Complete a safe boot up, login, logout, and shutdown sequence.

Expected Learning Outcomes:

- Students navigate the cursor smoothly and execute clicks accurately.
- Students maintain an upright typing posture with elbows at 90 degrees.
- Students log in and turn off computers using proper software shutdown, not the physical power button.
- Students define basic terminology: Desktop, Folder, Icon, Taskbar, Pointer, Booting.

Topics Covered

- Holding the mouse: Ergonomics, pointer/middle finger placement.
- Mouse actions: Single click (selection), right-click (context menu), double-click (opening), drag-and-drop, scrolling.
- Touchpad usage: Gestures, left/right clicks on laptops.
- Keyboard layout: Alphanumeric keys, Spacebar, Enter, Backspace, Delete, Shift, Caps Lock, Arrow keys.
- Typing posture: Spine alignment, home row concept, hand relaxation.

- Power procedures: Booting process, logging in, user sessions, logging out, shutting down.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Mouse Practice games: Playing bubble-popping or simple block puzzle games that require clicking and dragging.
- Typing drills: Practicing home row keys (ASDF JKL;) on software like Tux Typing.
- Boot & Shut down cycles: Performing the complete power-on, login, logout, and software shutdown cycle under teacher guidance.

Student Activities:

- Partner drills: One student types a short sentence while the other observes and reports posture alignment.
- Desktop exploration: Finding icons on the desktop, moving them around, and organizing them in straight columns.

Teacher Guide & Notes

Watch students carefully during early keyboard and mouse practice. Intervene immediately when you see students slouching, using only one hand for the keyboard, or holding the mouse like a claw. Explicitly demonstrate why pressing the physical power button to turn off a running computer can corrupt files.

Exercises & Review Questions

Exercises:

- Identify the key: Which key moves the cursor to the next line?
- Fill in: The process of starting a computer is called _____.

Review Questions:

- Describe the correct way to hold a computer mouse.
- What is the difference between backspace and delete keys?
- What are the steps to safely turn off a computer?

Projects & Assessments

Mini Project:

The Typing Challenge: Complete a 3-minute typing test aiming for at least 15 Words Per Minute (WPM) with 90% accuracy using home row finger placements.

Assessment Method:

Observational assessment: Individual grading of a student performing desktop actions (drag-drop, double-click) and shutting down correctly.

Home Assignment:

Write a checklist of the rules for correct typing posture in your notebook.

Required Materials

Typing software, mouse pads, computers, posture wall posters.

Module 4: Operating Systems

Module Overview

This module introduces students to software, specifically the Operating System (OS). It compares Windows, macOS, and Linux. Students will learn how to navigate desktop interfaces, search for files, change system settings, manage user accounts, install/uninstall basic programs, and execute simple troubleshooting steps.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Explain the purpose of an Operating System.
- Recognize the visual layouts of Windows, macOS, and Linux OS.
- Navigate the Start Menu, Taskbar, Settings, and Notifications panel.
- Install and uninstall basic applications safely.

Expected Learning Outcomes:

- Students explain that the OS manages hardware and software.
- Students search for and open applications (e.g. calculator, paint).
- Students locate system updates, adjust screen brightness, and volume.
- Students execute basic troubleshooting (rebooting, checking power connections).

Topics Covered

- What is an Operating System? The middleman between user and computer.
- Major Operating Systems: Microsoft Windows, Apple macOS, Linux.
- The Desktop environment: Start Menu/Dock, Taskbar, system tray, notifications.
- System Settings: Display settings, volume, Wi-Fi, time/date, background wallpaper.
- Application management: Searching, launching, installing, and uninstalling software.
- System updates: Why they are important and how to run them.
- User accounts: Administrator vs. standard user accounts, basic password setups.
- Basic troubleshooting: What to do when a program freezes (Force Quit, Task Manager, Restart).

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Settings exploration: Changing desktop wallpaper, setting local time, and adjusting system volume.
- Software installation simulation: Installing a free app (e.g., VLC Media Player) and then safely uninstalling it.
- Task Manager demo: Intentionally freezing a program and using Task Manager (Ctrl+Shift+Esc) to force close it.

Student Activities:

- Navigating through 'Settings' to find the system specifications (CPU type, RAM size, OS version) and writing them down.
- Creating a second user account named 'Guest' and setting a custom profile picture.

Teacher Guide & Notes

Focus primarily on Windows since it is the most common OS in administrative settings, but show screenshots or a live demo of Linux and macOS. Introduce Task Manager early so students know they can resolve freezes without pulling the power cord.

Exercises & Review Questions

Exercises:

- Match OS logos to their correct names.
- Identify the tool: Which tool is used to force-close frozen programs?

Review Questions:

- What does an Operating System do?
- How do you check if your computer is connected to the internet in the Windows OS?
- What is the difference between an Administrator account and a Standard user account?

Projects & Assessments

Mini Project:

Custom OS Profile: Create a personalized desktop workspace. Set a neutral background wallpaper, pin three favorite applications to the taskbar, and organize desktop shortcuts.

Assessment Method:

Practical exam: Students are given 5 minutes to change screen resolution, create a guest account, pin an app, and open the system settings.

Home Assignment:

Write a 5-step guide in your notebook on how to install and safely uninstall a computer application.

Required Materials

Computers running Windows, projectors, printed sheets showing different OS interfaces.

Module 5: File Management

Module Overview

This module teaches the vital skill of file organization. Students learn file structures, creating folders, naming conventions, moving files, using trash/recycle bin, identifying common file extensions, and using portable USB flash drives and cloud storage systems.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Understand file system hierarchy (drives, folders, subfolders, files).
- Perform core file operations: Create, Rename, Copy, Move, and Delete.
- Identify common file extensions (.txt, .docx, .xlsx, .pdf, .jpg).
- Transfer files using USB drives and understand cloud storage concepts.

Expected Learning Outcomes:

- Students build organized, structured directories.
- Students use keyboard shortcuts (Ctrl+C, Ctrl+V, Ctrl+X, Ctrl+Z) for file management.
- Students recover mistakenly deleted files from the Recycle Bin.
- Students safely insert, transfer files to, and eject USB drives.

Topics Covered

- File storage concepts: The digital filing cabinet (Files vs. Folders).
- Directory structures: Main folders, subfolders, file paths.
- File Explorer / Finder navigation.
- Basic actions: Create, Rename, Delete, Restore from Recycle Bin.
- Advanced actions: Copy vs. Move (Cut), Paste, Drag-and-drop between directories.
- File Extensions: Text (.txt), Word (.docx), Excel (.xlsx), Images (.jpg, .png), Audio (.mp3), PDF (.pdf).
- Removable media: USB drives, SD cards, external hard disks, safe ejection.
- Cloud Storage: Google Drive, OneDrive, Dropbox overview.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Directory Creation challenge: Build a folder structure for school subjects: 'SchoolWork' > 'Math', 'Science', 'English' > 'Notes', 'Assignments'.
- Shortcuts Speedrun: Using only the keyboard (Ctrl+C, Ctrl+V, Ctrl+X) to move a file from one folder to another.
- USB Transfer: Saving a document to a local folder, copying it to a USB drive, ejecting the USB, inserting it into another computer, and reading the file.

Student Activities:

- Creating files, saving them with appropriate extensions, and categorizing them into folders.
- Restoring 3 files from the Recycle Bin and placing them back in their correct locations.

Teacher Guide & Notes

Emphasize the difference between copying (duplicating) and moving/cutting (shifting location). Demonstrate the Recycle Bin as a safety net, but explain that USB files are deleted permanently. Show how to safely eject drives to prevent data corruption.

Exercises & Review Questions

Exercises:

- Match extensions to file types (e.g. .jpg -> Photo).
- What shortcut is used to 'Undo' a file movement?

Review Questions:

- Why is organizing files in folders useful?
- What is the difference between copy-paste and cut-paste?
- Why should you 'safely eject' a USB drive before unplugging it?

Projects & Assessments

Mini Project:

The File Clean-Up: Given a folder containing 20 miscellaneous files (photos, documents, sheets), sort them into organized folders within 3 minutes.

Assessment Method:

Practical test where the teacher checks if the student can build a nested folder structure and transfer a file to a USB drive.

Home Assignment:

Write down the keyboard shortcuts for Copy, Paste, Cut, Undo, and Select All in your notebook.

Required Materials

Computers, USB flash drives, mock disorganized directories preloaded on student computers.

Module 6: Internet Basics

Module Overview

This module introduces students to the World Wide Web. It covers web browsers, search engines (specifically Google), safe search queries, cyber hygiene, online etiquette, email setup, and digital citizenship.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Explain the difference between the Internet and the World Wide Web.
- Navigate a browser, use tabs, bookmarks, and search bars.
- Construct effective search engine queries using keywords.
- Identify phishing signs, unsafe websites, and practice safe password hygiene.
- Compose and send formal and informal emails.

Expected Learning Outcomes:

- Students launch Google Chrome or Mozilla Firefox and type URLs directly.
- Students search for educational resources using specific search phrases.
- Students design strong passwords incorporating letters, numbers, and symbols.
- Students send an email with an attachment to the teacher.

Topics Covered

- What is the Internet? What is a Web Browser? (Chrome, Firefox, Edge, Safari).
- Search engines: Google, Bing, Yahoo. Keyword-based searching.
- URL structure: HTTP vs. HTTPS, domain names (.com, .org, .edu, .gov).
- Web safety: Safe browsing, avoiding pop-ups, checking for lock icons (HTTPS).
- Cybersecurity basics: Phishing emails, malware warnings, strong password design.
- Email: Setting up accounts, composing, subject lines, CC, BCC, attachments, spam folder.
- Digital Citizenship: Online etiquette, cyberbullying awareness, protecting personal information.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Information Hunt: Students search Google for answers to educational trivia questions (e.g., 'What is the height of Mt. Everest?') and copy the source link.
- Password Creator: Drafting a strong password, testing its strength on safe password checkers, and writing down storage tips.
- Email Drafts: Setting up a student email account, composing an introductory message, attaching an image, and sending it to the teacher.

Student Activities:

- Creating bookmarks of 3 useful educational websites (e.g., Wikipedia, Khan Academy).
- Roleplaying scenarios: Reviewing printed printouts of emails to decide if they are 'Spam/Phishing' or 'Real'.

Teacher Guide & Notes

Emphasize cyber safety heavily. Students aged 12-18 are vulnerable online. Teach them never to share personal details (address, phone number, school) online. Review email etiquette, explaining that formal communication rules apply to emails sent to teachers/employers.

Exercises & Review Questions

Exercises:

- Identify secure URLs: Which URL is secure: http://example.com or https://example.com?
- What is the purpose of the 'BCC' field in an email?

Review Questions:

- What is the difference between a web browser and a search engine?
- Name three rules for creating a strong, secure password.
- What is digital citizenship?

Projects & Assessments

Mini Project:

The Cyber Safety Pledge: Students create a personalized Digital Citizenship Pledge detailing how they will stay safe, protect their privacy, and treat others kindly online.

Assessment Method:

Evaluation of email project: Students must send an email with the correct subject line, structured greetings, and a text file attachment.

Home Assignment:

Write down a list of 5 internet safety rules that should be posted in every school computer lab.

Required Materials

Internet access, web browsers, email template sheets, safe password templates.

Module 7: Microsoft Office Suite

Module Overview

This module provides a comprehensive introduction to the three primary productivity applications in Microsoft Office: Word (word processing), Excel (spreadsheets), and PowerPoint (presentations). It also covers OneDrive/Google Drive and handling PDF documents.

Estimated Duration

4 Weeks (8 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Create, edit, format, and save documents in Microsoft Word.
- Use tables, bullet lists, page numbers, and images in Word documents.
- Identify cells, rows, columns, and enter data in Microsoft Excel.
- Apply basic formulas (SUM, AVERAGE, multiplication) and create charts in Excel.
- Build multi-slide presentations with animations and graphics in PowerPoint.
- Convert files to PDF and upload them to cloud drives.

Expected Learning Outcomes:

- Students write and format formal letters and reports in Word.
- Students calculate school grade averages and household budgets using formulas in Excel.
- Students design and present slide shows on school topics in PowerPoint.
- Students save Office documents as PDFs and upload them to Google Drive/OneDrive.

Topics Covered

- Microsoft Word: Interface (Ribbon), font styling, alignment, headers/footers, margins, tables, page layouts, inserting images, printing.
- Microsoft Excel: Columns, Rows, Cells, Cell ranges, entering text/numbers, cell formatting, basic formulas (addition, subtraction, multiplication, division, SUM, AVERAGE), basic bar/pie charts.

- Microsoft PowerPoint: Theme selection, slide layouts, adding text/images, slide transitions, basic object animations, presenter view.
- Cloud Sync: Saving to OneDrive or Google Drive, sharing links, PDF conversions.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Word Report: Typing a 1-page report on 'My Favorite Historical Figure', inserting their photo, styling headings, and formatting body text.
- Excel Budget: Creating a table showing 5 monthly expenses, listing values, calculating the sum, and displaying the expenses as a pie chart.
- PowerPoint Slide Deck: Designing a 5-slide presentation on a topic of interest, applying a slide transition, and delivering it to a peer.

Student Activities:

- Creating a table of contents in a Word document.
- Entering class test marks into an Excel sheet, sorting them, and using =AVERAGE to find the mean score.
- Designing a PowerPoint presentation on 'Computer Hardware' to show to the class.

Teacher Guide & Notes

Dedicate 1.5 weeks to Word, 1.5 weeks to Excel, and 1 week to PowerPoint. In Excel, make sure students write actual formulas (e.g., =A1+B1) rather than calculating values manually and typing the result. Emphasize clean, readable slides over cluttered, high-animation decks.

Exercises & Review Questions

Exercises:

- In Excel, what symbol must every formula start with?
- What menu option in Word is used to insert a picture?

Review Questions:

- What is the difference between Microsoft Word and Microsoft Excel?

- How do you save a Word document as a PDF?
- Why are slide transitions used in PowerPoint presentations?

Projects & Assessments

Mini Project:

The Small Business Tracker: Create a Word invoice, an Excel inventory list with formulas, and a PowerPoint sales pitch deck for a fictional lemonade stand business.

Assessment Method:

A timed, multi-part practical assessment requiring students to create a formatted document, a spreadsheet with formulas, and a presentation.

Home Assignment:

Create a list of 5 situations where Excel is better than Word, and 5 situations where Word is better than Excel.

Required Materials

Computers with Microsoft Office (Word, Excel, PowerPoint) installed, templates for invoices and budgets.

5. Phase 2: INTERMEDIATE (Creative Logic & App Development)

Introduction & Context

Welcome to Phase 2 (INTERMEDIATE) of the Digital Literacy & Computer Education Programme. Having established fundamental hardware and operating system literacy in Phase 1, students are now introduced to the creative and analytical world of software development. This phase moves students from passive consumers of applications to creators of technology. They will learn computational thinking, algorithmic logic, visual block programming in Scratch, text-based programming in Python, foundational web programming with JavaScript, and get introduced to mobile application development with React Native. The core pedagogy focuses on logical problem solving, decomposition of complex tasks, debugging, and interactive creations.

Phase Description

Phase 2 spans 16 weeks, representing 32 lessons of 45 minutes each. It bridges the gap between basic literacy and advanced technology. The curriculum starts with language-independent logic (Algorithms & Flowcharts), then visualizes these concepts using Scratch, followed by the text-based syntax of Python. Students then learn client-side scripting logic using JavaScript before finalizing with a conceptual and hands-on introduction to building mobile apps. Projects are central to this phase: students build games, math engines, and mobile screens to develop practical development experience.

Learning Objectives & Core Outcomes

Learning Objectives:

- Develop logical thinking, problem-solving, and algorithmic skills using flowcharts and pseudocode.
- Design and build interactive animations, games, and stories in Scratch using loops, events, and variables.
- Write, test, and debug clean, readable Python code utilizing inputs, operators, loops, and data structures.
- Explain the role of JavaScript in web browsers and manipulate basic web elements (DOM basics).

- Understand the framework architecture of React Native and design simple UI components for mobile screens.

Expected Learning Outcomes:

- Students can break down a complex daily task into a step-by-step flowchart algorithm.
- Students create multi-sprite interactive games in Scratch with automated scoring systems.
- Students write console scripts in Python to perform mathematical calculations, sort data, or execute text games.
- Students write simple JavaScript functions that change text or colors on click in web simulations.
- Students build mock layouts for a Calculator, Notes app, and Quiz app in React Native.

Core Skills Students Will Gain

- Algorithmic formulation and logical parsing
- Flowchart mapping and pseudocode writing
- Visual program execution and message broadcasting
- Python syntax debugging and syntax analysis
- Control flow manipulation (loops, branch logic)
- Data structure usage (variables, lists, dicts)
- Event-driven web scripting
- Cross-platform mobile UI design concepts

Weekly Schedule

Weeks	Module Title	Topics Overview
17-18	Module 8: Computational Thinking & Algorithmic Logic	Decomposition, pattern recognition, flowcharting, pseudocode, debugging strategies.

19-22	Module 9: Scratch Visual Programming	Sprites, blocks, motions, variables, loop control, conditionals, broadcast events, game creation.
23-27	Module 10: Python Programming Basics	Variables, types, operators, if-else, while/for loops, functions, lists, dictionaries, projects.
28-30	Module 11: JavaScript Concepts & Web Logic	Syntax, variables, loops, browser events, DOM selection, simple web manipulations.
31-32	Module 12: App Development (React Native Intro)	React Native structures, component design (Text, View, Image), mock apps (Notes, Calculator, Quiz), Swift awareness.

Recommended Teaching Methods & Classroom Activities

Teaching Methods:

- Live Code-Alongs: Teachers write code on a projector, explaining their logic while students type along.
- Predictive Exercises: Showing students a piece of code and asking them to predict the output before running it.
- Debug-A-Thons: Intentionally inserting errors into files and letting students compete to find and fix them.
- Unplugged Activities: Using physical cards, grid maps, and human-instructions to explain loops and conditionals.
- Project-Based Learning: Building incremental components of a game or utility across multiple classes.

Classroom Activities:

- Human Robot Walk: One student acts as a 'robot' and must navigate a maze using only precise commands from a partner.
- Scratch Animation Showdown: Designing custom animations with custom sprites and sound files.
- Python Math Wizard: Writing a console script that takes user age and calculates their age in dog-years or seconds.
- Interactive Web Buttons: Coding simple JavaScript functions that change page background colors on click.

Hands-on Practical Activities

- Flowcharting Daily Tasks: Creating a step-by-step flowchart detailing the process of crossing a busy street.
- Scratch Catching Game: Building a game where a player catches falling objects to score points before a timer runs out.
- Python Calculator Console: Coding a Python terminal application that adds, subtracts, multiplies, or divides two numbers.
- React Native Notes Screen: Coding a styled React Native layout that displays a list of draft notes with icons.

Projects Framework

Individual Project:

The Python Math & Text Game Engine: Students design and code a Python console application from scratch. The application must offer a menu to the user: 1. Play a Number Guessing Game (using random number generators and conditionals), 2. Run a Simple Calculator, or 3. Run a Text-Based Adventure Story. The project requires the student to use variables, loops, functions, and standard input/output models.

Group Project:

Interactive Educational Game: Working in pairs, students design and build an educational game in Scratch (e.g. a math quiz game or a geography sorting puzzle). One student designs the visual layout, custom sprites, and menus, while the other implements the

scoring loops, variable logic, and state changes. They present their completed game to the class.

Teacher Guidance & Lab Notes

> **Facilitator Pacing Note**
> *Teacher Note: Transitioning from block-based coding (Scratch) to syntax-based coding (Python) is a major hurdle. Syntax errors (missing colons, indentation errors) will frustrate students. Teach them that debugging is a natural, positive part of programming. Emphasize logic over memorizing syntax; they can always check documentation, but logic must be developed.*

Required Physical & Digital Resources

- Computers (minimum 2GB RAM) with Python 3 and Node.js installed
- Offline Scratch Desktop editor or internet access to Scratch online
- Text Editors: Visual Studio Code, Thonny (highly recommended for Python beginners), or IDLE
- Expo Go app installed on student/school smartphones (for React Native previews)
- Flowcharting tools (Draw.io, whiteboard, or grid paper)

Assessment Methods & Framework

- Weekly Code Challenges: Small programming tasks to evaluate comprehension of loops or functions.
- Code Reviews: Students explain their logic line-by-line to the teacher or peers.
- Project Evaluation Rubric: Measuring code organization, logic completeness, and functionality.
- End-of-Phase Practical Exam: Developing a working script under timed classroom settings.

Grading Rubrics

Component	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Algorithmic Logic	Decomposes complex problems cleanly; creates accurate, logical flowcharts.	Flowcharts are logical with minor notation or connection errors.	Struggles with decomposition; flowcharts have logical errors.	Cannot construct a logical flowchart or outline program logic.
Scratch Coding	Uses events, loops, variables, and messages; builds smooth, interactive projects.	Uses basic logic; projects run correctly with minor design bugs.	Struggles with loops or variables; code is repetitive.	Cannot create a working Scratch project.
Python Scripting	Writes clean Python code; uses correct indentation, functions, loops, lists.	Writes working Python scripts; minor syntax or indentation errors.	Struggles with loops or functions; code crashes frequently.	Cannot write basic syntax-correct Python code.
JS & DOM Basics	Understands syntax; writes functions that interact with browser DOM elements.	Writes basic JS; minor logic errors; can link script to HTML.	Struggles with syntax or event-listeners; basic code runs.	Cannot write basic JavaScript.
Mobile App Concepts	Develops structured React Native layouts;	Understands React Native basics; layouts	Struggles with styles/components; concepts	Unable to outline mobile app

	understands Views, Text, Styles.	are functional.	are unclear.	components.

End-of-Phase Examination Framework

Phase 2 Final Examination Structure:
Part A: Concept Check (20 Marks)
- 10 Multiple Choice Questions (algorithms, variable types, loops, events)
- 5 Code Output Prediction Questions (Students read code snippet and write output)
- 5 Debugging Questions (Find syntax/logical errors in written snippets)

Part B: Practical (80 Marks)
- Task 1: Computational Logic (15 Marks): Create a detailed flowchart for a login screen validation algorithm.
- Task 2: Python Scripting (35 Marks): Code a console program that requests a list of student names, stores them, asks the user to search for a name, and outputs whether the student is present or absent.
- Task 3: JavaScript DOM Event (15 Marks): Write HTML and JavaScript code where clicking a button labeled 'Dark Mode' toggles the page background color between white and gray.
- Task 4: React Native Layout (15 Marks): Write code structure for a React Native Screen containing a Header, an Input text box, and a Button styled using stylesheet properties.

Certification Requirements

Certification of Basic Programming Competency:
To be awarded the Certificate of Basic Programming Competency, a student must meet the following criteria:
1. Minimum attendance of 85% in all Phase 2 programming labs.
2. Successful delivery of the Individual Game Engine and Group Educational Game with functional code and documentation.
3. Achieve a minimum score of 55% on the End-of-Phase Programming Examination.

Module 8: Computational Thinking & Algorithmic Logic

Module Overview

This module introduces students to the foundational logic of computer science before writing code. Students learn the four pillars of computational thinking: Decomposition, Pattern Recognition, Abstraction, and Algorithmic Design. They will design flowcharts and pseudocode to solve everyday problems and learn basic debugging techniques.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Break down a complex problem into smaller, manageable parts (Decomposition).
- Identify patterns and rules in problems to create templates (Pattern Recognition).
- Design step-by-step algorithms to solve mathematical and logical problems.
- Draw flowcharts using correct standard shapes and symbols.

Expected Learning Outcomes:

- Students decompose everyday processes (e.g. buying lunch, cleaning a room).
- Students define flowchart symbols (Terminator, Process, Decision, Input/Output).
- Students write step-by-step instructions (pseudocode) for sorting objects.
- Students find logical errors in a faulty set of instructions.

Topics Covered

- The Four Pillars of Computational Thinking: Decomposition, Pattern Recognition, Abstraction, Algorithmic Design.
- What is an Algorithm? Step-by-step logic.
- Flowchart symbols and standards: Oval (Start/End), Rectangle (Process), Diamond (Decision), Parallelogram (Input/Output), Arrows.
- Writing Pseudocode: English-like programming logic.
- Logic branches (Yes/No decisions) and loops (repeating actions until a condition changes) in algorithms.
- Introduction to debugging: Syntax errors vs. logical errors.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Human Maze: Students draw grid flowcharts on paper and direct classmates through physical mazes using only those commands.
- Flowchart Drawing: Drawing algorithms for a calculator program or password system on whiteboards or paper.

Student Activities:

- Decomposing the recipe of their favorite meal into a precise step-by-step algorithm.
- Drawing a flowchart representing the logic: 'If it is raining, take an umbrella. Else, wear sunglasses. Walk to school.'

Teacher Guide & Notes

Do not touch computers in the first week. Use 'unplugged' activities. Help students see that coding is just expressing algorithms in programming syntax. Emphasize logical rigor: computers only do exactly what they are told, nothing more.

Exercises & Review Questions

Exercises:

- Identify what each flowchart shape represents.
- Given a simple pseudocode, walk through its execution step-by-step on paper.

Review Questions:

- What is abstraction in computational thinking?
- Draw and name the four core flowchart shapes.
- What is the difference between a syntax error and a logical error?

Projects & Assessments

Mini Project:

The Sorting Algorithm: Design a flowchart and pseudocode representing how a robot would sort 5 numbers from smallest to largest.

Assessment Method:

Written quiz testing computational thinking vocabulary and a drawing test requiring students to flowchart a login validation system.

Home Assignment:

Write a 10-step algorithm for brushing teeth. Ensure you handle decisions, such as 'Is toothpaste tube empty?'

Required Materials

Whiteboards, markers, flowchart templates, grid paper.

Module 9: Scratch Visual Programming

Module Overview

Scratch is a visual, block-based programming language developed by MIT that teaches coding logic without syntax errors. Students learn sprite creation, blocks, motions, loops, conditions, variables, and events. They will build simple interactive stories, animations, and arcade games.

Estimated Duration

4 Weeks (8 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Navigate the Scratch interface (Stage, Sprites, Block Palette, Scripts Area).
- Use motion, look, sound, and pen blocks to manipulate sprites.
- Implement control logic using repeat loops, infinite loops, and conditional blocks (If-Then).
- Create and manipulate variables to track scores, health, and timers.

Expected Learning Outcomes:

- Students build programs by dragging and snapping logical blocks.
- Students create games with custom character assets.
- Students program interactive events, such as keypress triggers or mouse clicks.
- Students use broadcast messages to coordinate action between multiple sprites.

Topics Covered

- Scratch Interface: Sprite manager, Stage, Backdrop library, Costumes.
- Events: 'When green flag clicked', 'When key pressed', Sprite clicks.
- Motions: Steps, coordinate plane (X, Y), gliding, rotation, bouncing off walls.
- Looks & Sounds: Changing costumes, speech bubbles, sound effects.
- Control Logic: Repeat blocks, 'Forever' loops, 'If-Then-Else' conditionals.
- Sensing: Key input, sprite touching, edge detection.
- Data & Variables: Creating score counters, modifying variables.
- Messaging: Broadcaster and receiver blocks.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Animate a Name: Students select letters of their names as sprites and program them to spin, change color, and play sounds on click.
- Chase Game: Programming a cat sprite to follow the mouse pointer while a mouse sprite runs away, scoring points when caught.

Student Activities:

- Building a conversation between two sprites using broadcast and delay blocks.
- Adding a high score counter and a countdown timer to a Chase Game.

Teacher Guide & Notes

Scratch is highly engaging. Ensure students don't spend all their time drawing costumes; they must write logic blocks. Show them how Scratch block logic coordinates directly with the algorithms and flowcharts they drew in Module 8.

Exercises & Review Questions

Exercises:

- Explain what a 'forever' block does.
- Identify the category (Motion, Look, Control, Sensing) of various blocks.

Review Questions:

- What is a sprite in Scratch?
- How do loops help prevent duplicate code in Scratch?
- What block is used to change a character's appearance?

Projects & Assessments

Mini Project:

The Maze Solver Game: Build a game where a player navigates a maze using the arrow keys, restarting if they touch the black walls, and winning if they reach the treasure chest.

Assessment Method:

Rubric grading of the student's Scratch maze game, analyzing their code structure and logic completeness.

Home Assignment:

Write a 5-step description of how your game logic handles the event when a sprite touches a specific obstacle.

Required Materials

Scratch Desktop installed or web access, mouse devices.

Module 10: Python Programming Basics

Module Overview

Python is a popular, clean, text-based programming language. This module introduces programming syntax, variables, data types, console input/output, operators, conditions, loops, functions, lists, and dictionaries. Students write simple console-based projects to solve problems.

Estimated Duration

5 Weeks (10 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Explain the difference between compiler/interpreter, and execute Python code in Thonny.
- Declare and use variables of different types (Integers, Floats, Strings, Booleans).
- Write conditional statements using if, elif, and else.
- Control loops using for and while structures.
- Write reusable blocks of code using functions and list/dictionary data structures.

Expected Learning Outcomes:

- Students write scripts that take user input and display formatted text output.
- Students solve mathematical equations using standard operators in Python.
- Students write loops to print sequences or traverse structures.
- Students construct simple applications (e.g. password checkers, quiz consoles).

Topics Covered

- Introduction to Python: Syntax, Thonny IDE, print() function, comments.
- Variables and Data Types: integer (int), decimal (float), text (str), boolean (bool).
- User Input: input() function, type casting (int(), float(), str()).
- Arithmetic and Comparison Operators: +, -, *, /, %, >, <, ==, !=.
- Conditional Logic: if, elif, else statements.
- Loops: while loops (for conditions) and for loops (iterating over ranges).
- Functions: def keyword, parameters, return values.

- Data Structures: Lists (creating, accessing indexes, appending) and Dictionaries (key-value basics).

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Interactive Shell practice: Executing simple math and string actions in Python REPL.
- Console calculator: Coding a program that asks for two numbers and a math operator, and outputs the result.
- Number Guessing Game: Building a game where Python picks a random number and the user guesses, getting hints ('Too high', 'Too low') until correct.

Student Activities:

- Debugging a list of intentionally broken python scripts provided by the teacher.
- Writing a function that takes a name and returns a customized welcome string.

Teacher Guide & Notes

Indentation is the most common error source in Python. Emphasize using tabs/spaces consistently. Use visual traces: write code on the board and show how variables change state step-by-step. Introduce type conversion (casting input to integer) early to avoid runtime math errors.

Exercises & Review Questions

Exercises:

- Identify syntax errors in a given code snippet.
- State the final value of variable X after a given loop runs.

Review Questions:

- Why do we need to cast variables (e.g. int(input()))?
- What is the difference between a list and a dictionary?
- How do parameters work in Python functions?

Projects & Assessments

Mini Project:

The Gradebook Analyzer: Write a program that asks the teacher to input 5 grades, saves them in a list, calculates the average grade, and outputs the highest and lowest scores.

Assessment Method:

Code submission and walkthrough: Students submit a script for grading and explain their function structures to the teacher.

Home Assignment:

Write a Python script on paper that prints the numbers 1 to 20, but for multiples of 3, print 'Fizz' instead.

Required Materials

Thonny IDE installed, python syntax sheets, sample project templates.

Module 11: JavaScript Concepts & Web Logic

Module Overview

JavaScript is the programming language of the web. In this module, students learn basics of JavaScript syntax (variables, functions, loops, conditions) and explore how JavaScript interacts with HTML to create dynamic, interactive webpages.

Estimated Duration

3 Weeks (6 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Understand the role of JavaScript in web development (compared to HTML/CSS).
- Write basic JavaScript syntax, declare variables, and write functions.
- Connect a JavaScript script to an HTML document using script tags.
- Understand DOM (Document Object Model) selection and manipulate element properties.

Expected Learning Outcomes:

- Students write JavaScript console logs and alerts.
- Students code functions triggered by HTML button clicks.
- Students modify text content on a web simulator using JavaScript.
- Students write simple verification rules for form inputs.

Topics Covered

- Introduction to JavaScript: Browsers, script tags, console.log(), alert().
- Syntax basics: let, const, variables, basic arithmetic.
- Functions: declaring functions, arguments, handling events.
- Events: onclick, onchange, mouseover, page load events.
- DOM Basics: document.getElementById(), document.querySelector().
- Manipulating DOM: changing innerHTML, altering style properties (colors, display properties).

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Greeting Builder: HTML page containing an input field for a student name and a button. Clicking the button updates the page text to 'Hello, [name]!' using JavaScript.
- Color Changer: JavaScript function that alters document.body.style.backgroundColor based on button clicks.

Student Activities:

- Adding double-click events to image elements to toggle visibility.
- Writing a simple validation script that checks if a text input field is empty before showing an alert.

Teacher Guide & Notes

Keep JavaScript simple. Do not dive into complex objects, asynchronous functions, or APIs. Focus on the interactive feedback loop: user does something -> event triggers -> JavaScript executes -> DOM updates.

Exercises & Review Questions

Exercises:

- What command is used to select an HTML element with ID 'header' in JavaScript?
- True or False: JavaScript is only used to style text fonts.

Review Questions:

- How does JavaScript differ from HTML?
- What is an event-listener?
- What is the DOM in web browser terms?

Projects & Assessments

Mini Project:

The Web Counter: Build a webpage with a header showing number '0' and two buttons: '+' and '-'. Use JavaScript to increment or decrement the count value displayed.

Assessment Method:

Practical checklist: Students create a button that updates page content and styling dynamically on screen.

Home Assignment:

Write down 3 examples of websites you use that utilize JavaScript to update information without reloading the page.

Required Materials

Web browsers, text editors, basic HTML file templates.

Module 12: Mobile App Development (React Native Intro)

Module Overview

This module introduces students to the architecture of mobile application development. Using React Native, they will explore how code runs on smartphones. They will design basic user interfaces, build layouts for common utilities (Calculator, Notes, Quiz), and gain a brief awareness of native iOS Swift coding.

Estimated Duration

2 Weeks (4 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Explain the difference between mobile web apps, cross-platform apps, and native apps.
- Understand the role of React Native in compiling JavaScript into native mobile structures.
- Identify core React Native components (View, Text, Image, ScrollView, TextInput, TouchableOpacity).
- Create layouts using CSS Flexbox configurations in mobile layouts.

Expected Learning Outcomes:

- Students describe the difference between React Native (cross-platform) and Swift/Kotlin (native).
- Students construct UI layouts for a mobile screen using View and Text components.
- Students style layouts with custom padding, background colors, and centering properties.
- Students preview interactive mock UI designs on a mobile device using Expo Go.

Topics Covered

- Mobile development landscape: Native iOS (Swift), Native Android (Kotlin), Cross-Platform (React Native, Flutter).
- What is React Native? JavaScript code running native interface components.
- Core components: View (like div), Text (like p), Image, TextInput, ScrollView.
- Styling in React Native: Style objects, Flexbox layouts, margins, alignments.

- Interactive buttons: TouchableOpacity, onPress events.
- Preview tools: Expo framework, scanning QR codes to run apps on devices via Expo Go.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Expo Setup & Hello World: Initializing a basic React Native template, launching it, and viewing 'Hello World' on a mobile screen via Expo Go.
- Calculator Layout: Creating a grid layout of button components matching a pocket calculator.
- Quiz UI: Building a mobile question screen containing a progress bar, image area, and multiple choice options.

Student Activities:

- Designing a paper layout (wireframe) of a personal profile mobile app before coding its structure.
- Styling a login screen layout containing input fields, logo placeholders, and a primary button.

Teacher Guide & Notes

This module is conceptual and layout-focused. Don't worry about complex state management. Focus on getting UI elements visible on mobile screens. If computers or smartphones are limited, use web-based simulators like Snack (snack.expo.dev) which run React Native layouts entirely in the browser.

Exercises & Review Questions

Exercises:

- What React Native component is used to group other elements together?
- State which language is used to build native iOS applications exclusively.

Review Questions:

- What is React Native, and why is it useful for developers?

- What component is used to capture typed input from a user in React Native?
- Why is Flexbox important for mobile app styling?

Projects & Assessments

Mini Project:

The Quick Notes UI: Design a 1-page mobile notes layout. The screen must feature a header, a scrollable list showing 3 note entries with titles, summaries, and dates, and a floating '+' button at the bottom.

Assessment Method:

Evaluation of UI designs in snack.expo.dev: Grading visual layout consistency, alignment, component choices, and styling.

Home Assignment:

Write a 3-paragraph research report comparing React Native to native Swift in terms of deployment ease, learning speed, and device performance.

Required Materials

Computers with Thonny/VS Code, Expo account, smartphones with Expo Go, paper worksheets for UI sketches.

6. Phase 3: ADVANCED (Specialized Technologies & Futures)

Introduction & Context

Welcome to Phase 3 (ADVANCED) of the Digital Literacy & Computer Education Programme. This final phase transitions students into specialized, modern technological domains. Students will learn professional front-end web design with responsive HTML and CSS, version control and collaborative engineering using Git and GitHub, and explore the principles of Artificial Intelligence and responsible prompt engineering. They will also delve into Robotics and physical computing (using Arduino microcontrollers), media creation tools (Canva, video, audio), cybersecurity hygiene, and career mapping in tech. The objective of this phase is to make students creatively independent, secure, and ready for further academic studies or professional opportunities.

Phase Description

Phase 3 spans 22 weeks, representing 44 lessons of 45 minutes each. It covers advanced, real-world topics. The curriculum provides hands-on practice: students build portfolio sites, collaborate on code repositories on GitHub, construct physical electronic projects, edit media assets, and practice cybersecurity defenses. Given the technical nature of physical computing, specific warnings regarding funding and low-cost alternatives are integrated. Career awareness modules outline educational pathways and skills for 12 core computer science professions.

Learning Objectives & Core Outcomes

Learning Objectives:

- Build fully functional, semantic HTML/CSS portfolios that adjust to mobile, tablet, and desktop screens.
- Utilize Git and GitHub command lines or desktop apps to commit, branch, clone, push, and collaborate on code.
- Formulate structured prompts using advanced techniques, verify AI statements, and understand AI ethics.
- Assemble and program basic physical computing circuits using Arduino, LEDs, sensors, and resistors.

- Develop basic digital marketing assets using Canva, edit audio files, and perform basic video editing cuts.
- Identify security risks (phishing, social engineering, malware) and implement protective hygiene.
- Create detailed educational and skills roadmaps for core careers in technology.

Expected Learning Outcomes:

- Students publish a personal responsive portfolio website online using GitHub Pages.
- Students collaborate on a group web design project, resolving branch conflicts in Git.
- Students use AI tools (ChatGPT/Gemini) productively as research assistants while verifying citations.
- Students program an Arduino circuit that flashes an LED when light levels drop below a sensor threshold.
- Students design a brand kit in Canva and edit a 1-minute video explaining a technological concept.
- Students practice secure data backup rules and create simulated threat-modeling diagrams.
- Students present a complete path-to-employment roadmap for their chosen technology career.

Core Skills Students Will Gain

- Semantic web development and responsive layout structures
- Branching, committing, and conflict resolution in Git
- Effective AI prompting and secondary source verification
- Basic electrical circuit design and embedded C programming
- Graphic design aesthetics and raster/vector layout creation
- Audio-video media compiling and editing techniques
- Threat identification and mitigation strategies in cybersecurity
- Professional presentation and career pathway planning

Weekly Schedule

Weeks	Module Title	Topics Overview
1-4	Module 13: Web Design (HTML & CSS)	Semantic HTML5, CSS layout engines, responsive web styling, portfolio layouts, CSS Flexbox/Grid.
5-7	Module 14: Git & GitHub Collaboration	Local vs. remote repos, commits, staging, push/pull, cloning, branching, conflict resolution, GitHub Pages.
8-10	Module 15: AI & Responsible Prompting	LLMs, prompt engineering, ChatGPT/Gemini productivity, limitations, AI ethics, citation verification.
11-13	Module 16: Robotics & Physical Computing	Arduino introduction, sensors, circuits, LEDs, breadboards, programming loops, budget hardware notes.
14-16	Module 17: Digital Creativity & Media	Canva layout basics, graphic styling, audio editing, video timelines, compiling video/audio assets.
17-19	Module 18: Cybersecurity & Safe Practices	Viruses, malware, ransomware, phishing vectors, social engineering, encryption basics, password managers.
20-22	Module 19: Career Awareness & Mapping	Skills and educational pathways for 12 core tech careers, CV drafting, portfolio

	compiling, final wrap-up.

Recommended Teaching Methods & Classroom Activities

Teaching Methods:

- Project-Based Portfolio Construction: Guide students to build a single site that grows with each lesson.
- Collaborative Coding Simulations: Split classes into software groups that push to the same GitHub repository.
- Interactive AI Labs: Presenting prompts, analyzing AI hallucinations, and cross-referencing statements online.
- Physical Lab Assemblies: Working in small teams with hardware boards, matching breadboard diagrams.
- Guest Panels and Roleplaying: Simulating technical job interviews or security breach drills.

Classroom Activities:

- The Git Conflict Hack: Students pair up, modify the same file line, commit, and walk through resolving the merge conflict.
- AI Fact-Checking Race: Teacher inputs a fabricated question into an AI, and students compete to find the accurate sources.
- Robotic Light Controller: Assembling an Arduino board with a light-dependent resistor that sounds a buzzer when it gets dark.
- Security Threat Analysis: Reviewing case studies of famous digital hacks to identify the point of entry and suggest fixes.

Hands-on Practical Activities

- School Website Layout: Designing a responsive layout structure for a modern school homepage.
- GitHub Project Publication: Committing a web design project locally, pushing to GitHub, and enabling GitHub Pages.

- Canva Brand Campaign: Creating a set of 3 styled social media cards promoting a local non-profit company.
- Career Pathway Presentation: Creating a slide deck showing the qualifications, skills, and timelines to become an AI Engineer.

Projects Framework

Individual Project:

Personal Portfolio Website & Repository: Students design and code a responsive personal portfolio webpage using semantic HTML5 and CSS. The site must contain an 'About Me' section, a list of completed projects with links, and a contact form. They must track this project using Git, commit changes regularly, push it to a remote GitHub repository, and publish the site live using GitHub Pages.

Group Project:

The Robotics Lab Showcase (or Virtual Simulation): Working in groups of three, students construct a physical project using Arduino (such as an automatic plant watering indicator or a traffic light simulator). If physical hardware is unavailable, they build the project virtually using TinkerCAD Circuits. They document their circuit diagram, write the control code, upload it to GitHub, and demonstrate the working model.

Teacher Guidance & Lab Notes

> **Facilitator Pacing Note**
>
> *Teacher Note: Module 16 (Robotics) requires hardware funding. If the school does not have a budget to purchase Arduino boards, sensors, breadboards, and jumpers, DO NOT skip this module. Use free, web-based simulators like TinkerCAD (tinkercad.com/circuits) which allow students to drag physical parts and write code that simulates identically to real hardware. This ensures no student is left behind due to resource limits.*

Required Physical & Digital Resources

- Text Editors: Visual Studio Code or Sublime Text
- Git CLI and GitHub Desktop installed, student GitHub accounts
- Internet browsers with web tools (Inspect Element)

- Arduino Starter Kits: Arduino Uno board, breadboard, LEDs, resistors (220 ohm, 10k ohm), LDR, jumper wires, servo motor
- Alternative: Web browsers with TinkerCAD accounts
- Digital Creativity software: Canva (free educational plan), Audacity (audio), Shotcut/CapCut (video)

Assessment Methods & Framework

- GitHub Repo Inspection: Evaluation of student commit logs to check regularity and clarity of descriptions.
- Interactive Project Walkthroughs: Direct student presentations of web layouts and physical circuits.
- Conceptual Written Exam: Core concepts of cybersecurity, git actions, AI, and career pathways.
- Portfolio Web Review: Inspection of CSS styling, media queries, and code organization.

Grading Rubrics

Component	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Web Design (HTML/CSS)	Semantic structures; layout is responsive on phone and PC; clean CSS styles.	Functional structures; layout responds with minor layout defects on small screens.	Basic layout styling; page layout breaks on small mobile viewports.	Cannot write basic CSS layout or HTML structure.
Git & GitHub Usage	Smooth commits, branches, pushes, and pulls; resolves	Commits and pushes files regularly; knows basic commands;	Can execute basic clone/commit but requires help to push or	Cannot use Git or set up repositories.

	conflicts; publishes live Pages.	struggles with conflicts.	merge.	
AI & Prompting	Constructs advanced prompts; critiques AI; cross-references and verifies text details.	Understands prompt guidelines; gets good results; uses basic verify routines.	Uses AI as a simple search search; accepts text without verification.	Unable to build structured prompts.
Robotics & Circuits	Builds safe circuits; writes clear C loops to interact with sensors; codes inputs/outputs.	Builds basic circuits; basic code loop runs; struggles with sensor calculations.	Can assemble basic LED circuit but struggles with code and sensors.	Cannot build circuit or write Arduino code.
Cybersecurity Basics	Understands threats; identifies phishing, malware, privacy parameters; maps protections.	Understands basic threats; uses basic hygiene; lacks detailed strategy.	Struggles to identify basic threat patterns or social scams.	Cannot define cyber safety rules.

End-of-Phase Examination Framework

Phase 3 Final Examination Structure:

Part A: Theory (30 Marks)

- 10 Multiple Choice Questions (git actions, responsive design, AI terminology, basic circuitry)
- 10 Fill in the blanks (cybersecurity vectors, career skill definitions)

- 10 Short Answer Questions (e.g., Explain the role of a media query in responsive CSS design)

Part B: Practical (70 Marks)
- Task 1: Web Design (25 Marks): Design a responsive contact section with a header, name/email input form, and button, styled using a CSS layout media query that wraps elements vertically on screens under 600px.
- Task 2: Git Control (15 Marks): Initialize a local folder, commit the HTML file, create a branch named 'dev', modify code, commit on 'dev', merge back to 'main', and write out commands used.
- Task 3: Arduino Simulation (15 Marks): Draw the schematic or write the code block for a circuit that turns on an LED (pin 13) only when a light sensor (analog pin A0) reads a value below 300.
- Task 4: Career Mapping & Review (15 Marks): Draft a cover letter for IT Support Specialist detailing required skills.

Certification Requirements

Certification of Advanced Digital Literacy & Technology Concepts:
To be awarded the Certificate of Advanced Digital Literacy, a student must meet the following criteria:
1. Minimum attendance of 85% in all Phase 3 classes and labs.
2. Successful completion and publication of the Personal Portfolio Website on GitHub Pages.
3. Submission of the Robotics project/simulation and Career Presentation with passing grades.
4. Achieve a minimum score of 60% on the End-of-Phase Advanced Examination.

Module 13: Web Design (HTML & CSS)

Module Overview

This module teaches professional front-end web design. Students will write semantic HTML5 structures and apply visual layout styles using CSS. They will learn layout models (Flexbox, Grid) and construct responsive designs that scale to mobile and desktop viewports using CSS media queries.

Estimated Duration

4 Weeks (8 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Write valid, semantic HTML5 tags (header, nav, main, section, footer, article).
- Apply CSS rules, selectors, colors, margins, padding, and text styling.
- Understand layout models: Block, Inline, Flexbox, and CSS Grid.
- Use CSS media queries to design responsive layouts that scale to small screens.

Expected Learning Outcomes:

- Students write code structures that pass browser layout tests.
- Students separate content (HTML) from presentation (CSS) in separate files.
- Students style layouts with custom color palettes and grid columns.
- Students design multi-page web layouts that are readable on mobile phones.

Topics Covered

- Web architecture: How browsers render HTML. Semantic tags.
- CSS basics: Selectors (tag, class, ID), properties, the Box Model (Margin, Border, Padding, Content).
- CSS Colors: HEX, RGB, HSL. Font styling and Google Fonts integration.
- CSS Layouts: Display properties, Float vs Flexbox, Grid configurations.
- Responsive Web Design: Viewport meta tags, percentage widths, media queries.
- Structuring projects: Asset folders, relative image/link paths.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Bio Page Styling: Creating a personal biography webpage, adding style classes for headings, borders, and margins.
- Flexible Photo Gallery: Building a card layout of photos that displays as 4 columns on PC, 2 columns on tablet, and 1 column on mobile screens using Flexbox.

Student Activities:

- Creating navigation bars that collapse or adjust orientation depending on screen width.
- Applying consistent styling tokens (variables or style rules) for buttons and headings.

Teacher Guide & Notes

Focus heavily on the Box Model, as margins and padding confuse beginners. Teach mobile-first concepts: design for small screens first, then use media queries to scale up. Emphasize using inspect element in browsers to debug CSS layout lines.

Exercises & Review Questions

Exercises:

- Identify what 'margin: 10px 5px 10px 5px;' represents in top-right-bottom-left terms.
- Write the syntax for a media query that targets screens smaller than 600px.

Review Questions:

- What is the difference between an ID selector and a Class selector in CSS?
- What is semantic HTML and why is it important for accessibility?
- How does Flexbox help in aligning navigation items?

Projects & Assessments

Mini Project:

The School Homepage Layout: Code a homepage layout for a school featuring a header with navigation, an announcement section, an image gallery, and a styled contact footer.

Assessment Method:

Inspection of the personal webpage code structure, checking CSS selectors, validation errors, and mobile screen responsiveness.

Home Assignment:

Write a CSS style sheet that defines a navigation bar with a dark blue background, white text, and teal hover effects.

Required Materials

Web browsers, text editors, code cheatsheets.

Module 14: Git & GitHub Collaboration

Module Overview

Version control is crucial for software engineering. This module teaches Git and GitHub. Students will learn commands for tracking changes, committing files, pushing code online, branching, cloning repositories, collaborating on projects, and hosting static sites on GitHub Pages.

Estimated Duration

3 Weeks (6 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Explain the difference between Git (version control) and GitHub (hosting service).
- Initialize a local repository, stage files, and commit changes with clear descriptions.
- Push local code to a remote GitHub repository and clone existing code.
- Manage branches, perform merges, and resolve simple merge conflicts.

Expected Learning Outcomes:

- Students track changes in their projects using Git terminal or GUI.
- Students write meaningful, professional commit logs.
- Students publish static HTML portfolio websites to GitHub Pages.
- Students participate in code sharing and peer project contributions.

Topics Covered

- What is version control? Why is it important? Local vs. remote directories.
- Git installation, setup, user configurations (username, email).
- Command line operations: git init, git status, git add, git commit, git log.
- GitHub interaction: git remote add, git push, git pull, git clone.
- Branching: git branch, git checkout, git merge.
- Conflict resolution: What happens when code lines overlap, how to edit files to resolve issues.
- GitHub Pages: Hosting static web structures for free.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- First Commit: Initializing a local folder, creating 'index.html', staging, committing, and pushing it to a new GitHub repository.
- Conflict Resolution Drill: Partner exercise where two students clone the same repo, change line 5 of index.html to different text, push sequentially, and resolve the resulting merge conflict.

Student Activities:

- Cloning a teammate's project, adding a styled paragraph in a new branch, and submitting a pull request.
- Enabling GitHub Pages on their personal repository to share their live website.

Teacher Guide & Notes

Git is conceptual. Draw diagrams on the board showing commits as timeline points and branches as tracks. Use the concept of a 'Time Machine' to describe checking out old commits. Require all students to create clean GitHub accounts for their portfolios.

Exercises & Review Questions

Exercises:

- What command is used to check which files are modified but unstaged?
- What is the difference between 'git push' and 'git pull'?

Review Questions:

- What is a commit message, and why should it be descriptive?
- How does branching protect main production code?
- What is a merge conflict?

Projects & Assessments

Mini Project:

The Collaboration Portfolio: Working in pairs, create a project repository containing HTML profiles. Each student writes their own bio page in a separate branch, merges it to main, and publishes the combined page on GitHub Pages.

Assessment Method:

Teacher reviews the student's GitHub commit logs to evaluate formatting consistency, command usage, and project updates.

Home Assignment:

Write down the exact sequence of 7 Git commands required to initialize, commit, and push a local folder online.

Required Materials

Git installed, Internet, Git command sheets.

Module 15: AI & Responsible Prompting

Module Overview

This module introduces artificial intelligence (AI). It focuses on prompt engineering, using tools (ChatGPT, Gemini) for productivity and research, understanding limitations (hallucinations), AI ethics, and fact-checking statements.

Estimated Duration

3 Weeks (6 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Explain the concepts of Large Language Models (LLMs) and artificial intelligence.
- Write structured prompts using role definitions, context, and clear format constraints.
- Analyze AI outputs critically, identifying inaccuracies and 'hallucinations'.
- Discuss the ethical concerns of AI (plagiarism, bias, data privacy, environmental costs).

Expected Learning Outcomes:

- Students write prompts that yield precise, useful academic output.
- Students identify and document instances of AI error in research trials.
- Students cross-reference AI-generated claims against authoritative sources.
- Students use AI responsibly as a study tutor while maintaining academic integrity.

Topics Covered

- What is Artificial Intelligence? How do Large Language Models work (predictive text patterns)?
- Prompt engineering basics: Role, Task, Context, Constraints, Format.
- Iterative prompting: Refining outputs through follow-up conversations.
- Productivity use cases: Brainstorming, summarization, code debugging, writing outlines.
- Limitations: Hallucinations, outdated databases, bias in training datasets.
- Fact-checking: Navigating original citations, checking primary sources.
- AI Ethics: Copyright, academic integrity (plagiarism), data privacy (never inputting personal info).

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Prompting Battle: Students are given a complex output requirement (e.g. 'explain photosynthesis to a 6-year-old using baking metaphors'). They compete to see whose prompt yields the best response.
- Hallucination Spotting: Students prompt an AI to write a historical biography, then verify 5 specific statements (birth dates, achievements) using search engines to find mistakes.

Student Activities:

- Using AI to debug a broken Python script by inputting the code and error log, then testing the recommended fix.
- Creating a study schedule tailored to their subjects by prompting an AI with specific time limits.

Teacher Guide & Notes

Do not ban AI. Teach students how to use it as a powerful collaborator. Stress that AI is often confident but wrong. Make fact-checking a mandatory step. Emphasize academic honesty: using AI to write essays for submission is plagiarism.

Exercises & Review Questions

Exercises:

- Rewrite a weak prompt (e.g., 'tell me about frogs') into a structured prompt using the Role-Task-Context model.
- True or False: LLMs understand facts like a human.

Review Questions:

- What is a hallucination in AI terms?
- Why is inputting personal data or passwords into public AI engines dangerous?
- What are three indicators that an AI-generated claim might be incorrect?

Projects & Assessments

Mini Project:

The AI Study Assistant: Design a custom prompt template that turns an AI tool into an interactive quiz tutor. The prompt must instruct the AI to ask one multiple choice science question at a time and explain the answer only after the student replies.

Assessment Method:

Students submit a log containing their prompts, AI outputs, and documented verification steps for a chosen historical topic.

Home Assignment:

Write a 300-word essay discussing the environmental and social impacts of training large AI neural networks.

Required Materials

Internet access, accounts on ChatGPT or Google Gemini, prompt guides.

Module 16: Robotics & Physical Computing

Module Overview

This module bridges software and hardware. Students will learn the Arduino platform, simple circuit design, wiring breadboards, and writing basic code to control LEDs, motors, and read sensors. Due to funding considerations, low-cost options and virtual simulators (TinkerCAD) are detailed.

Estimated Duration

3 Weeks (6 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Understand the differences between microprocessors (e.g., Raspberry Pi) and microcontrollers (e.g., Arduino).
- Identify pins, power inputs, and components on an Arduino Uno board.
- Assemble electrical circuits on breadboards with resistors, LEDs, and sensors.
- Write basic C-based code loops using pinMode(), digitalWrite(), and analogRead().

Expected Learning Outcomes:

- Students read schematic wiring diagrams and assemble them.
- Students write code to flash LEDs, trigger buzzers, and control motors.
- Students read inputs from analog sensors (like light-dependent resistors).
- Students troubleshoot faulty wiring or code logic.

Topics Covered

- Introduction to physical computing: Microcontrollers vs. General computers.
- Arduino Uno board layout: Analog pins, Digital pins, Ground (GND), 5V/3.3V power, USB interface.
- Electrical basics: Voltage, Current, Resistance, Ohm's Law.
- Components: Breadboards, Jumper wires, Resistors (color bands), LEDs, Pushbuttons, LDR (Light sensor), Servo motors.
- Programming Arduino: Setup() and Loop() functions, digitalWrite(), delay(), pinMode(), analogRead().

- Virtual simulation: TinkerCAD Circuits interface.
- Budget note: Low-cost starter kits (clones of Arduino Uno cost under $10).

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Blinking LED: Building a circuit with an Arduino and an LED, writing code to turn the LED on and off every second.
- Light-Sensing Nightlight: Assembling an LDR circuit that measures room brightness and automatically turns on a warning LED when darkness is simulated.

Student Activities:

- Simulating a traffic light sequence using red, yellow, and green LEDs wired on a breadboard.
- Writing an Arduino script that changes the blink speed of an LED based on analog potentiometer dials.

Teacher Guide & Notes

If your school has no budget for physical kits, utilize TinkerCAD Circuits (tinkercad.com). It is completely free, runs in the browser, and simulates components accurately. Warn students about short circuits: always connect a resistor in series with LEDs to prevent burning them out.

Exercises & Review Questions

Exercises:

- Calculate resistance needed for a 5V circuit with 20mA LED limit using Ohm's Law (R = V/I).
- Identify the function of 'delay(1000);' in Arduino code.

Review Questions:

- What is a breadboard, and how are its sockets connected inside?
- Why is a resistor necessary when wiring an LED?

- What is the difference between digital pins and analog pins on the Arduino?

Projects & Assessments

Mini Project:

The Automated Burglar Alarm: Build a circuit (physical or virtual) featuring a sensor (motion or light) and a buzzer. Write code that sounds the alarm when the sensor state changes.

Assessment Method:

Students build and demonstrate a circuit. Grading evaluates correct wire alignments, resistor selection, and functional code loops.

Home Assignment:

Draw a wiring diagram of a circuit containing an Arduino Uno, a resistor, and a green LED connected to digital pin 9.

Required Materials

Arduino Uno kits or computer lab with TinkerCAD accounts, worksheets on circuit symbols.

Module 17: Digital Creativity & Media

Module Overview

This module introduces digital media creation. Students will learn graphic design basics using Canva, compile layouts, understand typography, and learn basic timeline video and audio editing techniques.

Estimated Duration

4 Weeks (8 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Apply basic design principles (contrast, alignment, hierarchy, balance) in digital layouts.
- Navigate the Canva workspace, use templates, layers, and export options.
- Perform basic audio edits, cuts, and noise reduction in Audacity.
- Compile, cut, splice, and export short video segments using standard timelines.

Expected Learning Outcomes:

- Students design promotional posters, social media grids, and report covers.
- Students edit voiceovers by removing noise and splicing audio tracks.
- Students edit short video packages with transitions, title cards, and backing tracks.
- Students choose appropriate media formats (.png, .mp4, .wav) depending on target displays.

Topics Covered

- Graphic Design principles: Color contrast, font selection, whitespace, visual hierarchy.
- Canva: Account setup, workspace navigation, templates, shapes, uploads, grouping elements.
- Media extensions: JPEG vs. PNG, Vector vs. Raster, MP3 vs. WAV, MP4 video compression.
- Audio Editing: Audacity workspace, importing tracks, recording voice, cut/copy/paste, noise reduction.
- Video Editing: Timeline concepts, tracks, slicing, transitions, adding titles, overlays, backing music, exporting.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Classroom Poster: Designing an educational infographic on computer parts using Canva and exporting it as a PDF print file.
- Audio Voiceover: Recording a 30-second speech about technology, editing it in Audacity to remove background noise, and adding a soft music track.
- Tech Explainer Video: Splicing three video clips, adding a title slide, inserting transitions, syncing the voiceover, and exporting as an MP4.

Student Activities:

- Designing a personal business card layout in Canva.
- Creating a short podcast segment by splicing together voice, sound effects, and transitions.

Teacher Guide & Notes

Students love creative modules. Ensure they maintain design discipline: avoid cluttering layouts with too many fonts or conflicting colors. Use free tools (Canva for Education, Audacity, CapCut/Shotcut) to keep access free.

Exercises & Review Questions

Exercises:

- State whether JPEG or PNG is better for transparent logos.
- Explain the concept of 'timeline tracks' in video editing.

Review Questions:

- What does 'visual hierarchy' mean in graphic design?
- How does noise reduction work in audio editing?
- Why is MP4 the most common format for web videos?

Projects & Assessments

Mini Project:

The Campaign Package: Create a digital media campaign for a fictional school club, including a Canva promotional poster, a 30-second audio advert, and a 1-minute video introduction.

Assessment Method:

Grading of creative portfolios based on alignment, design discipline, audio clarity, and editing continuity.

Home Assignment:

Write a 1-page script for a video tutorial explaining how to create a folder in Windows.

Required Materials

Computers with audio inputs, internet, Audacity and video editors installed, Canva accounts.

Module 18: Cybersecurity & Digital Hygiene Basics

Module Overview

This module teaches cybersecurity awareness and hygiene. Students will explore common threat vectors (phishing, malware, social engineering), learn about viruses, privacy settings, safe file downloads, and threat modeling.

Estimated Duration

2.5 Weeks (5 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Identify common digital threats, including computer viruses, malware, trojans, and ransomware.
- Recognize phishing emails, malicious links, and social engineering manipulation.
- Implement strong digital hygiene practices (password managers, two-factor authentication, safe downloads).
- Understand the basics of data encryption and private browsing modes.

Expected Learning Outcomes:

- Students analyze suspicious messages and flag threat indicators.
- Students configure browser security settings and verify software downloads.
- Students outline plans for secure regular offline and cloud backups.
- Students write threat-modeling tables for securing personal accounts.

Topics Covered

- The threat landscape: What is malware? Viruses vs. Trojans vs. Ransomware.
- Phishing: How attackers trick users. Malicious attachments and URL spoofs.
- Social Engineering: Impersonation, phone scams, and keeping passwords secret.
- Digital Hygiene: Strong passwords, two-factor authentication (2FA), password managers.
- Browser safety: Pop-up blockers, extensions, incognito mode limits.
- Software sourcing: Safe downloads vs. cracked/pirated software risks.
- Backup strategies: 3-2-1 backup rule (3 copies, 2 media types, 1 offsite).

Classroom & Hands-on Activities

Practical Hands-on Activities:

- Phishing Detector: Reviewing 10 sample emails (some real, some fake) to locate signs of fraud (misspelled domains, urgent requests, bad formatting).
- Backup Implementation: Simulating a backup process by copying important files to an external drive and setting up an automated sync to cloud storage.

Student Activities:

- Drafting a checklist of action items to secure a smartphone against physical and digital access.
- Writing a security advisory poster for their school warning classmates about local social media scams.

Teacher Guide & Notes

Cybersecurity is not just about code; it is mostly about human behavior. Focus on social engineering. Teach students to be healthily skeptical. Emphasize that 'free/cracked' software is the most common way secondary school students get malware on their home computers.

Exercises & Review Questions

Exercises:

- Identify the vulnerability: An email arrives from your bank requesting immediate password verification. What do you do?
- What does 2FA stand for and how does it protect accounts?

Review Questions:

- What is ransomware, and why is it dangerous for businesses?
- Explain the 3-2-1 backup rule.
- What are the limitations of private browsing (Incognito) mode?

Projects & Assessments

Mini Project:

The Lab Security Audit: Inspect the school computer lab under the teacher's guidance. Students list potential security vulnerabilities (e.g. passwords on notes, open ports, lack of screensavers) and write a remediation plan.

Assessment Method:

Written scenario test where students must write correct defense steps for various hacking attempts.

Home Assignment:

Write a 500-word essay about a major historical cyber breach, detailing the entry method, the impact, and how it could have been prevented.

Required Materials

Mock phishing emails, worksheets on security standards, backup drives.

Module 19: Career Awareness & Mapping

Module Overview

This final module prepares students for the future by mapping tech careers. It outlines educational pathways, certification programs, required skills, and portfolio development tips for 12 technology professions.

Estimated Duration

3 Weeks (6 Lessons)

Lesson Objectives & Learning Outcomes

Objectives:

- Identify the roles and responsibilities of 12 core technology professions.
- Understand the skill sets and tools associated with each technical career.
- Map educational pathways (university degrees, certifications, self-study resources).
- Prepare a basic professional resume and outline a portfolio strategy.

Expected Learning Outcomes:

- Students write summaries of their chosen future tech roles.
- Students identify relevant certifications (e.g., CompTIA, AWS, Cisco).
- Students write a professional education-and-skills roadmap.
- Students draft a standard CV and structure a project portfolio.

Topics Covered

- Career profiles: Software Engineer, Data Scientist, UI/UX Designer, AI Engineer, Cybersecurity Analyst, Cloud Engineer, Web Developer, Mobile Developer, Robotics Engineer, Network Engineer, Product Designer, IT Support Specialist.
- Required skills: Coding languages, tools, soft skills (communication, teamwork).
- Educational routes: Computer Science degrees vs. coding bootcamps vs. self-study.
- Industry Certifications: Entry-level certificates (CompTIA A+, Cisco CCNA, AWS Cloud Practitioner).
- Portfolio building: Documenting projects on GitHub, Behance, or personal websites.
- CV drafting and interview preparation basics.

Classroom & Hands-on Activities

Practical Hands-on Activities:

- My Dream Job Profile: Students select one tech career, research its salary, daily routine, and tools, and write a report.
- Roadmap Design: Drawing a visual career roadmap showing steps from age 16 to age 25, indicating schools, projects, and target companies.
- Mock Interview: Splicing class into interviewer/interviewee pairs to practice answering 'Tell me about a project you coded.'

Student Activities:

- Drafting a professional resume template including contact details, education history, and project links.
- Creating a list of 5 online courses (e.g. on Coursera, freeCodeCamp) that teach skills for their target job.

Teacher Guide & Notes

Inspire students but keep expectations realistic. Emphasize that tech careers require continuous learning. Show them that even without university access, self-study and portfolio projects on GitHub can lead to employment. Help them choose roles matching their strengths (e.g. artistic students toward UI/UX; analytical toward Data Science).

Exercises & Review Questions

Exercises:

- Match the job title to its primary tool (e.g. UI/UX Designer -> Figma).
- State what an IT Support Specialist does daily.

Review Questions:

- What is the difference between a Front-End Web Developer and a Back-End Web Developer?
- Why is a project portfolio often more important than a degree in the tech industry?

- Name three entry-level IT certifications.

Projects & Assessments

Mini Project:

The Personal Career Roadmap: Create a final poster or document mapping out a chosen tech career, listing 3 target skills, 2 entry-level projects they need to build, and 2 certifications they will study for.

Assessment Method:

Submission of the draft resume, career roadmap, and presentation delivery.

Home Assignment:

Find and write down the job requirements of a real junior tech position posted on a job search website.

Required Materials

Resume templates, career mapping worksheets, interview guide sheets.

7. Sample Lesson Plans

Below are five comprehensive, detailed sample lesson plans demonstrating how teachers should implement the curriculum modules across all three phases.

Lesson Plan 1: Mastering Keyboard & Typing Posture

Phase Context: Phase 1: Basic | Duration: 45 Minutes

Lesson Objectives

- Students will position their bodies in correct typing posture (back straight, feet flat, elbows at 90 degrees).
- Students will locate the 'Home Row' keys (ASDF JKL;) and place their fingers correctly.
- Students will type basic home-row character patterns without looking at the keyboard.

Required Lesson Materials

Computers, Tux Typing software, posture posters, keyboard worksheets.

Lesson Sequence & Activities

Introduction (10 Mins):
- Welcome students and seat them in the lab.
- Explain why posture is critical for health and typing speed.
- Demonstrate correct vs. incorrect sitting postures physically.

Main Lesson (15 Mins):
- Guide students to adjust their chairs and monitors.
- Introduce the Home Row concept. Explain that the left index finger goes on F (feel the tactile bump) and the right index finger goes on J (feel the bump).
- Project a keyboard layout showing finger-to-key associations.

Practical Exercise (15 Mins):
- Launch Tux Typing or Thonny to open an empty text file.
- Conduct a typing drill: Type 'a a a s s s d d d f f f' then 'j j j k k k l l l ; ; ;'.
- Circulate the room to correct physical hand setups.

Assessment & Homework (5 Mins):
- Exit Ticket: List the four letters your left hand rests on in the Home Row.

- Homework: Practice typing patterns on a printed keyboard layout sheet at home using your fingers.

Lesson Plan 2: Creating and Organizing Folders

Phase Context: Phase 1: Basic | Duration: 45 Minutes

Lesson Objectives

- Students will create new folders on the desktop and within directories.
- Students will rename folders using clear naming standards.
- Students will move files into folders using drag-and-drop and copy-paste methods.

Required Lesson Materials

Computers, preloaded sample text files, directory worksheets.

Lesson Sequence & Activities

Introduction (10 Mins):

- Use the analogy of a physical filing cabinet: files are papers, folders are cardboard sleeves.
- Explain what a folder directory is and why digital organization saves time.

Main Lesson (15 Mins):

- Demonstrate right-clicking on the desktop, selecting 'New > Folder', and naming it.
- Show how to double-click to open a folder and create a 'subfolder' inside it.
- Demonstrate moving a sample file into a folder using drag-and-drop.

Practical Exercise (15 Mins):

- Create a folder on the Desktop named 'MySchoolWork'.
- Inside it, create three subfolders: 'Math', 'Science', and 'English'.
- Move 3 sample text files from the Desktop into their matching subject folders.

Assessment & Homework (5 Mins):

- Check folders: Teacher visually checks student directories.
- Homework: List the steps required to rename an existing folder.

Lesson Plan 3: Python Logic & Conditional Branches (if-else)

Phase Context: Phase 2: Intermediate | Duration: 45 Minutes

Lesson Objectives

- Students will explain the logical role of conditional checking.
- Students will write basic Python scripts containing if-else branches.
- Students will evaluate comparisons (>, <, ==) in conditions.

Required Lesson Materials

Thonny IDE installed, logic card templates, projector.

Lesson Sequence & Activities

Introduction (10 Mins):

- Present real-world logic: 'If it is dark, turn on light. Else, turn off light.'
- Explain that conditional statements let programs make decisions based on changing data.

Main Lesson (15 Mins):

- Explain python if-else syntax, emphasizing the colon (:) and indentation blocks.
- Write a simple script checking if age is greater than 18:
  `if age >= 18: print('Adult') else: print('Minor')`.

Practical Exercise (15 Mins):

- Open Thonny IDE.
- Write a script that prompts the user to enter their test score (0-100).
- Use conditionals: If score >= 50, output 'Passed!'. Else, output 'Try Again!'.

Assessment & Homework (5 Mins):

- Code Check: Verify that scripts execute without indentation errors.
- Homework: Draw a flowchart representing a score checking algorithm.

Lesson Plan 4: Structuring Web Pages with HTML & CSS

Phase Context: Phase 3: Advanced | Duration: 45 Minutes

Lesson Objectives

- Students will build a structured webpage using semantic HTML5 tags.
- Students will link an external CSS style sheet to an HTML file.
- Students will style layouts using background colors, alignments, and padding.

Required Lesson Materials

VS Code, web browsers, CSS cheatsheets.

Lesson Sequence & Activities

Introduction (10 Mins):
- Explain that HTML provides structure (bones) and CSS provides styling (clothes/skin).
- Show how professional websites separate these files.

Main Lesson (15 Mins):
- Outline basic page setup: <!DOCTYPE html>, <html>, <head>, <body>.
- Introduce the <link> tag to attach styles.css.
- Demonstrate CSS class selectors and properties (color, background-color, font-family).

Practical Exercise (15 Mins):
- Create 'index.html' and 'styles.css' in a folder.
- Write a layout with a header, a section showing an image and paragraph, and a footer.
- Use CSS to style the header with a dark blue background, white centered text, and 20px padding.

Assessment & Homework (5 Mins):
- Browser review: Double-click HTML files to render them in browser tabs.
- Homework: Write a CSS class rule that centers text and adds a gray border.

Lesson Plan 5: Responsible Prompt Engineering & Verification

Phase Context: Phase 3: Advanced | Duration: 45 Minutes

Lesson Objectives

- Students will identify the key elements of a structured prompt.
- Students will use AI to assist research while spotting inaccuracies.
- Students will cross-reference claims using search engine resources.

Required Lesson Materials

Internet access, ChatGPT/Gemini interface, fact-checking worksheet.

Lesson Sequence & Activities

Introduction (10 Mins):

- Discuss what Large Language Models do: predict word matches based on training datasets.
- Explain that AI does not 'think' and can easily create false facts (hallucinations).

Main Lesson (15 Mins):

- Teach the Role-Task-Context-Format prompt template.
- Demonstrate prompting: 'Act as a historian. Summarize the invention of the telephone in 100 words. Format as bullet points.'
- Show a sample AI hallucination and explain how to verify it.

Practical Exercise (15 Mins):

- Prompt an AI tool to write an outline on the history of computers.
- Select 3 historical statements from the output.
- Search Google to find authoritative sources (museums, universities) confirming or correcting those points.

Assessment & Homework (5 Mins):

- Verification logs check: Students show links matching their verified details.
- Homework: Write down a structured prompt that outlines instructions for a study session.

8. Recommended Resources & Free Software

Teacher Support Resources

- Computer Science Teachers Association (CSTA) Standards & Guides - https://csteachers.org
- Code.org Teacher Panel - Curriculum resources and lesson plan slides - https://code.org
- Raspberry Pi Foundation Teacher Resources - Free physical computing plans - https://raspberrypi.org
- TinkerCAD Circuits Guides - Interactive simulation worksheets - https://tinkercad.com
- CS Unplugged - Free classroom activities teaching logic without computers - https://csunplugged.org
- Book: 'Computer Science Education: Perspectives on Teaching and Learning' by Sue Sentance
- Book: 'Coding in the Classroom: A Teacher's Guide to Helping Kids Code' by Ryan Somma

Student Learning Resources

- typing.com - Free typing drills and keyboard layout games
- w3schools.com - Interactive tutorials for HTML, CSS, JavaScript, and Python
- freecodecamp.org - Coding paths with interactive consoles
- Scratch online gallery - Explore and remix open-source projects - https://scratch.mit.edu
- sololearn.com - Mobile learning platform for Python and web design basics
- YouTube: 'Crash Course Computer Science' channel - Outstanding history and conceptual series
- YouTube: 'Programming with Mosh' channel - Highly structured beginner syntax guides

9. Appendices & Classroom Templates

Appendix A: Glossary of Core Computer Science Terms

Term	Definition
Algorithm	A step-by-step set of instructions or rules followed to solve a problem or complete a task.
Booting	The process of starting or restarting a computer system, loading the operating system into memory.
Browser	A software application used to access, retrieve, and view information on the World Wide Web (e.g. Chrome).
CPU	Central Processing Unit; the 'brain' of the computer that interprets and executes instructions.
CSS	Cascading Style Sheets; used to define the visual layout, styling, and design of web pages.
DOM	Document Object Model; a browser representation of the page structure, allowing scripting modification.
File Extension	A suffix at the end of a filename (e.g., .docx) indicating its format and which application opens it.
Hardware	The physical components of a computer system that you can touch (e.g. monitor, CPU).

HTML	HyperText Markup Language; the standard code language used to structure the content of webpages.
Incognito Mode	A browser privacy setting that prevents the storage of local history, cookies, or form data.
Input Device	Any hardware device that sends data to a computer, allowing user interaction (e.g. keyboard).
IPOS Cycle	Input, Processing, Output, Storage; the four core actions performed by all computer systems.
Operating System	The primary software that manages hardware resources and coordinates application programs.
Output Device	Any hardware device that receives data from a computer to display, print, or play (e.g. monitor).
Phishing	A cyberattack method where fraudulent emails trick users into sharing passwords or downloading malware.
Pseudocode	A simplified, English-like outline of a programming algorithm used before writing actual code syntax.
RAM	Random Access Memory; volatile, high-speed temporary storage used by the CPU for active processes.
Software	The set of digital instructions and programs that tell the hardware what to do (e.g. Microsoft Word).

Sprite	A two-dimensional image or character in visual programming (Scratch) that can be coded to move and interact.
URL	Uniform Resource Locator; the global address of a specific webpage or file on the internet.
Variable	A named storage container in programming used to hold and manipulate changing data values.

Appendix B: Essential Keyboard Shortcuts Cheatsheet

Shortcut	Operating System	Action Performed
Ctrl + C	Windows / Linux	Copy selected text, file, or folder to the clipboard.
Cmd + C	macOS	Copy selected text, file, or folder to the clipboard.
Ctrl + V	Windows / Linux	Paste copied text, file, or folder from the clipboard.
Cmd + V	macOS	Paste copied text, file, or folder from the clipboard.
Ctrl + X	Windows / Linux	Cut (move) selected text, file, or folder to the clipboard.
Cmd + X	macOS	Cut (move) selected text, file, or folder to the clipboard.
Ctrl + Z	Windows / Linux	Undo the last action performed.

Cmd + Z	macOS	Undo the last action performed.
Ctrl + S	Windows / Linux	Save the currently active document or file.
Cmd + S	macOS	Save the currently active document or file.
Ctrl + A	Windows / Linux	Select all items, files, or text in the active window.
Cmd + A	macOS	Select all items, files, or text in the active window.
Alt + Tab	Windows / Linux	Switch quickly between currently open applications.
Cmd + Tab	macOS	Switch quickly between currently open applications.
Ctrl + Shift + Esc	Windows	Launch the Windows Task Manager to monitor or force-close applications.
Cmd + Alt + Esc	macOS	Open the Force Quit Applications dialog menu.

Appendix C: Computer Laboratory Rules Poster

- 1. Food and Drinks: Absolutely no food or liquids are allowed near the computer workstations to prevent spills and hardware damage.
- 2. Hand Hygiene: Wash and dry hands before entering the lab to keep keyboards, mice, and touchscreens clean.
- 3. Equipment Handling: Handle all physical cables, mouse devices, keyboards, and boards gently. Never pull cords forcefully.

- 4. Startup/Shutdown: Always start and shut down computers using the operating system menu commands, never the physical power button.
- 5. Assigned Workstations: Work only at your assigned computer workstation. Do not change seats without permission.
- 6. Safe Storage: Save all your work in your designated student folder. Do not modify or delete files belonging to other students.
- 7. Authorized Software: Do not download, install, or run unauthorized applications, games, or browser extensions in the lab.
- 8. Lab Cleanliness: Pack away all jumper wires, components, and pens. Push your chair in and shut down your system before leaving.

Appendix D: Internet Safety & Cybersecurity Rules

- 1. Privacy Protection: Never share personal details online (full name, address, phone number, school, password pins).
- 2. Strong Passwords: Create passwords with at least 8 characters, combining uppercase/lowercase letters, numbers, and symbols.
- 3. Safe Downloads: Download files and software only from verified, trusted sources. Avoid cracked/pirated software domains.
- 4. Phishing Awareness: Do not click links or open attachments in emails from unknown senders. Verify sender addresses.
- 5. Secure Sites: Check for the lock icon and 'https://' in browser address bars before entering details on a website.
- 6. Cyberbullying: Treat others with kindness online. Report cyberbullying, threat messages, or mean comments to teachers.
- 7. Digital Footprint: Think before you post. Information posted online can remain permanent and affect your future career.

Appendix E: Digital Citizenship Pledge

> **Student Digital Compact**
> *Digital Citizenship Pledge*:
> *As a digital citizen of Dzuels Foundation, I pledge to*:

- Protect My Privacy: I will secure my passwords and keep my personal information private.
- Think First: I will verify information before sharing and avoid downloading unsafe files.
- Respect Others: I will communicate kindly and respectfully online, refusing to participate in cyberbullying.
- Support My Peers: I will collaborate positively and help my classmates learn computer science skills.
- Use Tech Safely: I will follow computer lab rules and care for hardware resources.
Student Signature: _____ Date: _____

Appendix F: Student Progress Tracking Template

Student Name	Phase 1 Status	Phase 2 Status	Phase 3 Status	Attendance %	Final Grade
John Doe	Completed (Pass)	Completed (Pass)	In Progress	92%	B+
Jane Smith	Completed (Pass)	Completed (Pass)	In Progress	95%	A
[Student Name]	[Pass/Fail/Audit]	[Pass/Fail/Audit]	[In Progress/Pending]	[Attendance %]	[Score Grade]

Appendix G: Laboratory Attendance Log Template

Date	Workstation ID	Student Name	Time In	Time Out	Teacher Sign
03/07/2026	Lab-04	John Doe	09:00 AM	09:45 AM	Approved
03/07/2026	Lab-08	Jane Smith	09:00 AM	09:45 AM	Approved

[DD/MM/YYYY]	[WS-ID]	[Student Name]	[HH:MM AM/PM]	[HH:MM AM/PM]	[Sign]

Appendix H: Rubric Grading Template

Criteria Checked	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)	Notes
Logic Accuracy	No errors	1-2 bugs	3-4 bugs	Crashing	Check loop indices
Code Comments	Thorough	Basic	Missing in parts	No comments	Add function headers
Formatting/Layout	Perfect spacing	Minor defects	Hard to read	No formatting	Check indent tabulations

Appendix I: Project Evaluation Template

Project Phase & Title	Lead Developer	Co-Developer(s)	Evaluation Date	Functional Score	Reviewer Comments
Phase 2: Catching Game	John Doe	Jane Smith	03/07/2026	95/100	Fantastic sprite scoring mechanics.
Phase 3: HTML Portfolio	Jane Smith	Solo	03/07/2026	98/100	Excellent CSS mobile-first queries.

Appendix J: Sample Certificate of Completion Structure

Official Certificate Template Layout

DZUELS FOUNDATION
Digital Literacy & Computer Education Programme

This is to certify that
[STUDENT NAME]
has successfully completed the requirements for the award of the
CERTIFICATE OF BASIC COMPUTER LITERACY
and is hereby certified competent in fundamental hardware operations, file organization, safe internet usage, and Microsoft Office productivity configurations.

Given under our hands this _____ day of _____, 20__.

_____ _____
Program Director *Lead Facilitator*

Appendix K: Dzuels Foundation Contact Information

Official Contact Points

For adoption queries, curriculum feedback, or sponsorship information, please contact:

Address: 12b Doherty Road, Ijero-Ekiti, Ekiti State, Nigeria
Phone: +234 707 664 6780, +234 904 249 3883
Email: info@dzuelsfoundation.org
Web: https://dzuelsfoundation.org